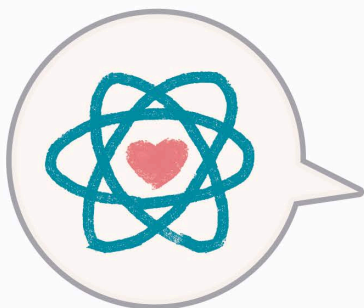


いあぐト!



第5版
React 19

TypeScript
で始める
つらくない
React 開発

③ React 実践編

大岡由佳

Next.js、React Router をサポート!

フレームワーク

FWの時代のReactを徹底解説
データ取得もフォームも怖くない

シリーズ累計

3.5万部
突破!

読者
からの声

「この本を選んだ自分を褒めてあげたい」
「実践的なReactを学びたい人にオススメ」

BOOTH
技術書部門

1位

りあクト！ TypeScriptで始めるつらくない React開発 第5版 【③ React実践編】

本書について

登場人物の紹介

柴崎雪菜（しばさき ゆきな）

都内のとあるインターネットサービスを運営する会社のテックリード・エンジニア。2016 年ごろから React を使っている。本格的なフロントエンド開発チームを作るための中核的人材として、今の会社に転職してきた。チームメンバーを集めるため人材採用にも関わり自ら面接も行っていたが、彼女の要求水準の高さもあってなかなか採用に至らない。そこで「自分が React を教えるから他チームのエンジニアを回してほしい」と上層部に要望を出し、社内公募が行われた。

秋谷香苗（あきや かなえ）

柴崎と同じ会社に勤務する、新卒 2 年目のやる気あふれるエンジニア。入社以来もっぱら Ruby on Rails によるサーバサイドの開発に携わっていたが、柴崎のメンバー募集に志願してフロントエンド開発チームに参加した。そこで柴崎から「1 週間で戦力になって」といわれ、彼女にマンツーマンで教えるを受けることになる。

前版との差分および正誤表

過去の版からの変更点と、本文内の記述内容の誤りや誤植についての正誤表は以下のページに掲載しています。なお、電子書籍版では訂正したものを新バージョンとして随時配信していきます。

- 各版における内容の変更
<https://github.com/klemiary/Riakuto-StartingReact-ja5.0/tree/main/CHANGELOG.md>
- 『りあクト！TypeScript で始めるつらくない React 開発第 5 版』正誤表
<https://github.com/klemiary/Riakuto-StartingReact-ja5.0/tree/main/errata.md>

サンプルコードについて

本文で紹介しているサンプルコードは、GitHub に用意した下記の専用リポジトリにて、章・節ごとに分けてすべて公開してあります。

<https://github.com/klemiuary/Riakuto-StartingReact-ja5.0>

挙動をその場で確認していただくため StackBlitz に置いているサンプルもありますが、本文中に随時 URL を掲載していますのでそちらをブラウザでご覧ください。学習を効率的に行うためにも、本文を読み進めながらこれらのコードを実際に実行することを強くおすすめします。またコードをご自身で変更して挙動のちがいを確かめてみるのも理解の助けになるはずです。

なおリポジトリのコードと本文に記載されているコードには、ときに差分が存在します。紙面に適切になるよう改行位置を調整したり、各種コメントアウト文を省略するなどによるものですが、そのため行番号が一致しないことがありますので、ご注意ください。

本文および上記リポジトリに掲載しているコードは、読者自身のプログラムやドキュメントに流用してかまいません。それにあたりコードの大部分を転載する場合を除き、筆者に許可を求める必要はありません。出典を明記する必要はありませんが、そうしていただければありがたいです。

本書内で使用している主なソフトウェアのバージョン

• React (react)	19.1.0
• React DOM (react-dom)	19.1.0
• Vite (vite)	6.3.5
• TypeScript (typescript)	5.8.3
• React Router (react-router)	7.6.0
• react-error-boundary (react-error-boundary)	6.0.0
• React Compiler (babel-plugin-react-compiler)	19.1.0-rc.1
• Conform (@conform-to/react)	1.6.0
• Zod (zod)	3.25.7

目次

本書について	2
登場人物の紹介	2
前版との差分および正誤表	2
サンプルコードについて	3
本書内で使用している主なソフトウェアのバージョン	3
第11章 React のページをルーティングする	8
11-1. React におけるルーティングとは	8
11-2. React 用のルーティングライブラリ	13
11-3. React Router を使う	15
11-3-1. React Router の特徴	15
11-3-2. React Router のセットアップ	17
11-3-3. React Router でのルーティング定義	26
11-3-4. React Router の各種 API	31
11-3-5. React Router をアプリケーションで使う（ライブラリモード）	43
11-3-6. React Router をアプリケーションで使う（フレームワークモード）	53
11-4. Next.js で App Router を使う	60
11-4-1. App Router のセットアップ	60
11-4-2. App Router でのルーティング定義	62
11-4-3. Server Components とキャッシュの話	65
11-4-4. Next.js ルーティングの各種 API	68
11-4-5. App Router をアプリケーションで使う	74
第12章 外部リソースのデータを取得する	78
12-1. 各種の外部データ取得手法	78
12-1-1. React における外部データ取得手法の変遷	78
12-1-2. アプローチからデータ取得手法を分類する	83
12-2. Fetch-on-Render アプローチ	84
12-3. Fetch-then-Render アプローチ	91
12-3-1. Server Components でのデータ取得	91
12-3-2. Route Loaders でのデータ取得	96
12-4. Suspense for Data Fetching	99
12-4-1. SPA での Suspense	99
12-4-2. Server Components と Suspense	106
12-4-3. Route Loaders と Suspense	110
第13章 エラーをハンドリングする	113

13-1. Error Boundary とは	113
13-2. react-error-boundary を使う	115
13-3. Next.js の App Router でエラーハンドリング	120
13-4. React Router (フレームワーク) でエラーハンドリング	124
第14章 メモ化でレンダリングを効率化する	130
14-1. 計算結果をメモ化する	130
14-2. 関数定義をメモ化する	134
14-3. React Compiler を導入する	138
第15章 フォームでデータを更新する	145
15-1. React でフォームの UI を実装するには	145
15-1-1. プリミティブな React フォーム	145
15-1-2. フォームのステータスを管理する	152
15-1-3. フォーム操作の UI への反映のタイミングをずらす	159
15-2. フォームを Action でハンドリングする	166
15-2-1. フォームの Action と Server Functions	166
15-2-2. Action にまつわる状態管理を行う	171
15-2-3. React Router の Route Actions を使う	175
15-3. フォームライブラリを利用する	183
15-3-1. フォームライブラリの選定	183
15-3-2. Conform の基本的な使い方	189
15-3-3. React Router (フレームワーク) で Conform を使う	197
15-3-4. Next.js で Conform を使う	210
エピローグ	214
あとがき	215

他巻目次

「① 言語・環境編」目次

第1章 こんにちは React

- 1-1. 基本環境の構築
- 1-2. React のプロジェクトを作成する
- 1-3. プロジェクトを管理するためのコマンドやスクリプト

第2章 ライトでディープな JavaScript の世界

- 2-1. あらためて JavaScript ってどんな言語？
- 2-2. 変数の宣言
- 2-3. JavaScript のデータ型
- 2-4. 関数の定義
- 2-5. クラスを表現する
- 2-6. 配列やオブジェクトの便利な構文
- 2-7. 式と演算子で短く書く
- 2-8. モジュールを読み込む
- 2-9. それでも知っておくべき JavaScript の this

第3章 関数型プログラミングでいこう

- 3-1. 関数型プログラミングは何がうれしい？
- 3-2. コレクションの反復処理
- 3-3. JavaScript で本格関数型プログラミング
- 3-4. 非同期処理と例外処理

第4章 TypeScript で型をご安全に

- 4-1. 時代が望んだ言語 TypeScript
- 4-2. TypeScript の基本的な型
- 4-3. 関数とクラスの型
- 4-4. 型の名前と型合成
- 4-5. さらに高度な型表現
- 4-6. 型アサーションと型ガード
- 4-7. モジュールのインポート／エクスポート
- 4-8. TypeScript の環境設定

「②React 基礎編」 目次

第 5 章 JSX で UI を表現する

5-1. なぜ React は JSX を使うのか

5-2. JSX 構文の書き方

第 6 章 React をめぐるモダン Web 開発の歴史（前編）

6-1. 第 1 世代：Ajax と DOM 操作のためのツールキット

6-2. 第 2 世代：SPA フレームワークの登場

6-3. パラダイムシフトを起こした React

第 7 章 ビルドツールでプロジェクトを運用する

7-1. React 開発におけるビルドツールの変遷

7-2. Vite で開発環境を構築する

第 8 章 コンポーネントの基本を学ぶ

8-1. UI コンポーネントライブラリの導入

8-2. コンポーネントに Props を受け渡す

8-3. コンポーネントに状態を持たせる

8-4. コンポーネントで副作用を扱う

8-5. React におけるコンポーネント構文の変遷

8-6. コンポーネントからロジックを分離し、再利用できるようにする

第 9 章 リンターとフォーマッターでコード美人に

9-1. リンターでコードの書き方を矯正する

9-2. フォーマッターでコードを整形する

9-3. Git Hooks と連携して実行させる

第 10 章 React をめぐるモダン Web 開発の歴史（後編）

10-1. 第 3 世代：SPA モデル黄金時代

10-2. 第 4 世代：メタフレームワークの時代

第 11 章 React のページをルーティングする

11-1. React におけるルーティングとは

「これまではページがひとつしかないサンプルしか出てこなかったけど、実際の開発にあたっては複数のページから構成されたアプリケーションを作ることになる。そのために必要になってくるのがルーティング（Routing）の機能。Web アプリケーションにおけるルーティングの定義とは、クライアントからのリクエストをその URL および HTTP メソッドに対応する処理に振り分けることを指す。Web アプリケーション構築に使われるルーティングシステムの役割を大きく分けると次の 3 つになるかな」

1. リクエストのパスに対してどのルーチンを結びつけるかのマッピング
2. ページ遷移やリダイレクトなどのナビゲーション制御
3. 現在のロケーション情報の取得

「このへんは Rails でもやることなのでわかります」

「そうだね。ところが Ruby on Rails で構築されるようなサーバサイドで稼働する MPA（Multi Page Application）¹と、React による SPA（Single Page Application）では、同じ機能を実現するにも実装方法を変える必要が生じてくる。その理由を理解するために、まず MPA でのルーティングの動作を図で見てみよう」

¹ 【② React 基礎編】の「6-1. 第 1 世代：Ajax と DOM 操作のためのツールキット」でふれています。

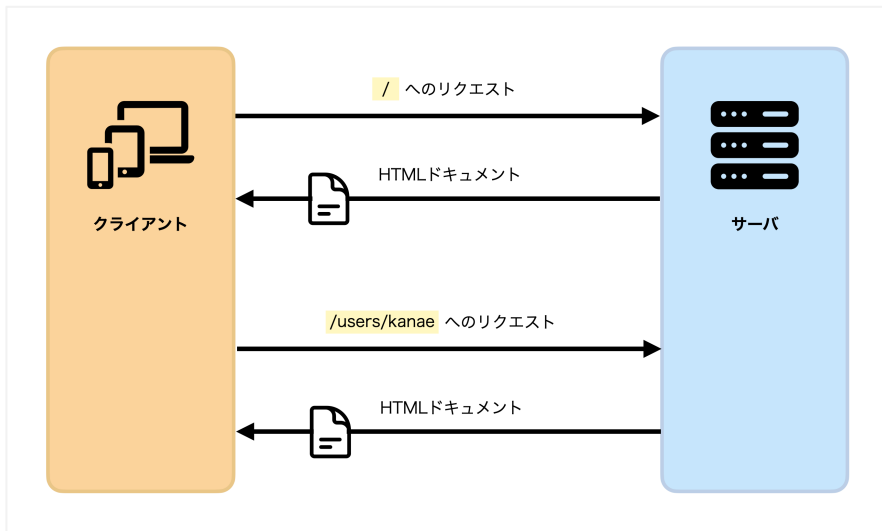


図 1: MPA のルーティング

「クライアントのブラウザがリクエストしたパスに対して、サーバが毎回それに対応した HTML ドキュメントを返す。非常に単純明快なくみだね。いっぽう、SPA でのルーティングの動作は次のようになる」

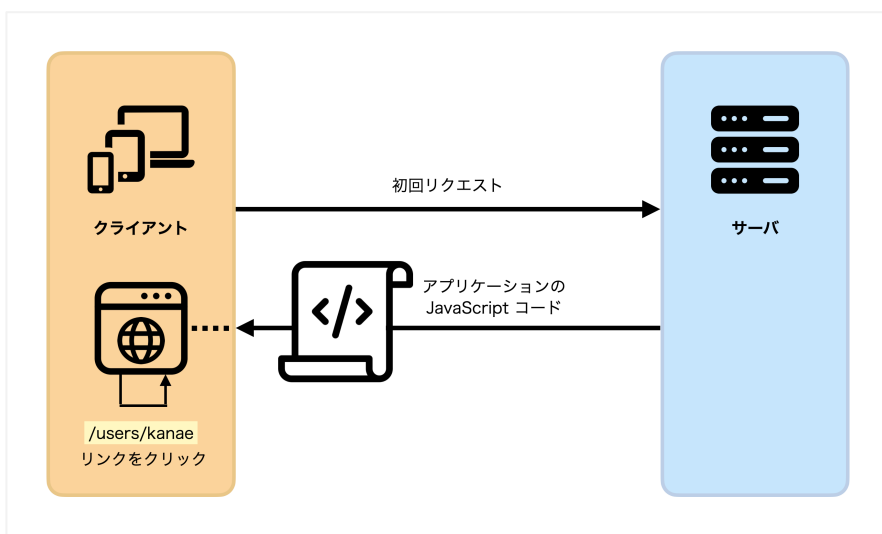


図 2: SPA のルーティング

「これはつまり、.....どういうことですか？」

「SPA では初回のリクエストに対して、そのリクエストのパスやメソッドに関係なくアプリケーション全体が記述された JavaScript コードが配信される²。そして JavaScript の解釈が終わると、リクエストパスに対応したページがレンダリングされる。そしてそれ以降のアプリケーション内でのページ遷移は、アプリケーションが動的に DOM 要素を書き換えることで実現される。ページ遷移によってブラウザのアドレスバーの URL が書き換わるがあっても、その処理はブラウザの中で完結していて実際にサーバへリクエストが飛ぶことは原則的にない。もちろん画像などのアセットファイルは別だけ」

「うーむ、SPA のルーティングってそうだったんですね。URL が変わってもサーバにリクエストが飛ばないというのは、まったく想像外でした」

「ちょっと Chrome でデベロッパーツールを開いて、『ネットワーク』のタブで『フィルタ』を『すべて』から『ドキュメント』に変更してみて。これで通常のサイトを見ると、ページ遷移時にドキュメントのリクエストだけが表示される。そのまま URL ボックスに `x.com` を入力して X (旧 Twitter) のトップページにアクセス。色んなところをクリックしてページ遷移してみて。URL は変わってるのに、ドキュメントのリクエストが一切ないでしょう？」

「ほんとだ。どのページに遷移しても `x.com` から追加も更新もされませんね」

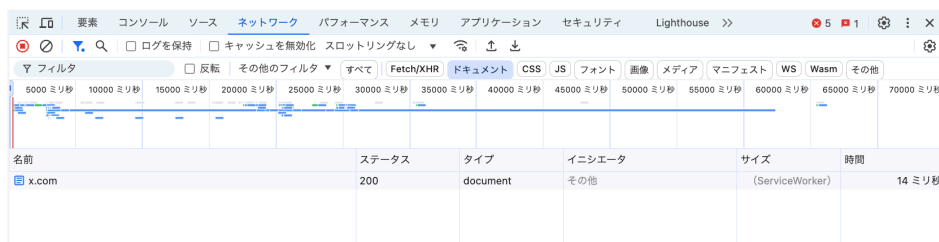


図 3: X サイト内でページを遷移してもリクエストが起らない

「これが SPA のルーティングの挙動なのね。URL の書き換えをとまなうページ遷移であっても、サーバにリクエストが行くことなく基本的にクライアントのブラウザ内で完結する。ただ、歴史的にここに至るまでにはワンクッションあったんだよね。秋谷さんは昔の Gmail や Twitter とかで #! みたいな記号が入った URL を見たことはない？」

「うーん、見た記憶はないと思います.....」

² ただしアプリケーションが「コード分割 (Code Splitting)」を用いてビルドされている場合、全体ではなくリクエストのパスが含まれた分割部分が配信される。そしてページ遷移時に他の分割部分が必要な場合、それがサーバからあらためて配信される。

<https://ja.legacy.reactjs.org/docs/code-splitting.html>

「む.....、最近の若い子は見たことないか。Web 標準 API にはブラウザ履歴を管理するための History API³ というものがあるんだけど、従来は前のページに戻る `back()` や次のページに進む `forward()`、履歴内の特定の位置に移動する `go()` くらいしかメソッドがなかった。そこで Ajax なアプリでサーバにいちいちリクエストを送ることなくルーティングを実現するために、URL に `#!`（※『ハッシュバン』もしくは略して『シバン』と発音する）を使う一種の裏技がいつとき流行ったの。URL に `#` ハッシュをつけるとページ内リンクアンカーになって、そこ以降が変わってもブラウザの履歴は変わるけどサーバにリクエストは飛ばないでしょ。それを裏技的に利用したわけ。

だから今の `twitter.com/yukishiba` みたいなユーザーページは、当時は `twitter.com/#!/yukishiba` のような URL で表現されてたのよね」

「へえ——、ちょっとした豆知識ですね」

「いや私にとっては実体験なんだけど.....、まあいいや。そういった経緯があって History API に後から `pushState()` と `replaceState()` というメソッドが実装された。前者がブラウザのセッション履歴のスタックに任意の項目を追加するもの、後者が現在の履歴を編集して任意の状態や URL で置き換えるものね。これによって JavaScript でブラウザの履歴が自由に書き換え可能になったわけ」

「それはいつごろの話なんですか？」

「仕様自体はもっと前からあったんだけど、モダンブラウザに実装され始めたのが 2009 年ごろから。標準仕様としては 2011 年に HTML5⁴ の仕様の一部としてドラフトが公開され、2014 年の最終勧告で正式に盛り込まれた。並行して AngularJS のようなフロントエンドフレームワークもルーティングに History API を使うようになっていった。いま現在、ほとんどすべてのフロントエンドフレームワークは、軒並みこの History API を使ってルーティングを実現してる」

「ふーむ、フロントエンドのルーティングにはそんな歴史があったんですね」

「こういったしくみの差から、アプリケーション内のリンク先の記述も MPA と SPA ではちがってくる。Laravel や Django ではテンプレートに直接 `<a>` タグを記述する。Ruby on Rails は少し特殊で `link_to` というヘルパーメソッドを使うけど、これもけっきょく `<a>` タグに変換されるので実質的には同じ。これをクリックするとサーバにリクエストが飛び、それに対応した新しい HTML ドキュメントが返され、受け取ったブラウザがそれをレンダリングすることでページ遷移が行われる。

いっぽう SPA では `<a>` を拡張した独自のリンク用 API を提供していて、アプリケーション内のリンク先の記述には `<a>` タグではなく必ずそれを使うことが求められる。SPA のルータはそのクリックイベントを拾ったうえで新しいページのレンダリングと同時に `pushState()` を実行する形で、クライ

³ 「履歴 API - Web API | MDN」 https://developer.mozilla.org/ja/docs/Web/API/History_API

⁴ W3C が策定した HTML の改訂第 5 版。2008 年にドラフトが発表され、2014 年に正式勧告、後に HTML5.2 まで勧告されたが、WHATWG が推す規格との対立から 2021 年 1 月に廃止され、HTML Living Standard が後継規格として HTML および DOM の標準とされた。HTML を最新のマルチメディア対応言語にすることを目的として、XHTML や DOM、ECMAScript の API が追加されていた。
<https://www.patterns.dev/vanilla/islands-architecture/>

アントに閉じたページ遷移を実現してるの」

「なるほどー。じゃ、リダイレクトなんかもちがうわけですね？」

「任意のリクエストに対してリダイレクトさせたい場合、MPA ではサーバが `Location` ヘッダに 300 番台のステータスコードを設定したレスポンスを返すことで、ブラウザがその `Location` に設定された URL にアクセスし直すという形になってる。いっぽう SPA では新しいページをレンダリングすると同時に `replaceState()` で現在の履歴をその URL に書き換える形で擬似的に実現される」

「『擬似的に』というのは？」

「たとえば検索エンジンのクローラは受け取ったステータスコードを解釈して、400 番台ならインデックスしないし 301 や 308 なら登録済みのインデックスを新しい URL に更新してくれたりする。ブラウザの挙動としては同じように見えてもサーバが適切にステータスコードを返してなければ、それは擬似的なものではないのよ」

「たしかにそうですね。だから実際のアプリケーション開発の場では、そういった挙動のちがいを意識しないといけないということでしょうか？」

「そのとおり。ただ最近の React ベースのメタフレームワークはサーバで稼働させることが増えてきてるので、SPA オンリーだった時代より問題が複雑になってきてる。Next.js の App Router では Server Components や Server Actions が、Remix や React Router v7 では Route Loaders や Route Actions がサーバで実行される。だからこういったフレームワークでは、ルーティング用の API もサーバで実行するものとクライアントで実行するものの 2 種類が用意されてるの。そのロジックが実行されるのがサーバなのかクライアントなのかを意識したうえで、それらを使い分ける必要がある」

「それはまた複雑ですね……」

「またこれらのフレームワークは、静的に生成したルートコンテンツをアプリケーションのバンドルに含めるため、初回アクセスより後のページ遷移でサーバにリクエストが飛ばないことも多々ある。これはアクセス解析やログ記録などを行いたいときには非常に都合が悪い」

「あー、そうですね。MPA なら一元的にサーバで管理できますけど。どうやって解決するんですか？」

「Google Analytics の場合を例にしよう。まずアプリケーションへの初回アクセスは必ずサーバにリクエストが来て、かつ SSR された HTML ドキュメントが送信されることになってる。だからその `<head>` にトラッキングコードを埋め込んでおく。これで初回ロード時のページビューは記録される。アプリケーション内のページ遷移は、ツリーのルート (root) に極力近いコンポーネントで `useEffect()` を使ってその依存配列にロケーション情報を格納することでルート (route) 変更を検知し、その副作用ルーチンの中で Google Analytics のサーバにページビューの情報を送信するようにしておく」

「うーん、けっこう面倒そう。新しく機能を作り込む際は、そうやって常にそれがサーバで動くかクライアントで動くかを考えて設計する必要があるんですね」

「そうね。でもだいじょうぶ。ちゃんと順を追って、わかるように教えてあげるから」

11-2. React 用のルーティングライブラリ

「React は本来、コンポーネントで UI を構築するためのライブラリなので、ルーティング機能を実現するためには別途ライブラリを組み合わせる必要がある。ルーティングライブラリは組み合わせる UI ライブラリ／フレームワークの構造に深く依存しているため、ほとんどの場合は専用のライブラリとして作られる。たとえば Preact は React と基本的なインターフェースに互換性を持たせた軽量な UI ライブラリだけど、React 用のルーティングライブラリをそのまま使うことはできない。Preact 版を出してるプロダクトもあるけど、オリジナルの React 版とはパッケージが分けられてる」

「そうなんです」

「名前も先につけたもの勝ち。だから Vue Router や Preact Router は本体の公式が出しているライブラリだけど、React Router は完全なサードパーティのライブラリで、単に React 向けのルーティングライブラリという以上の意味はない」

「React 用のルーティングライブラリってどんなものがあるんですか？」

「単体のルーティングライブラリとしては React Router が実績もユーザー数も圧倒的トップ。ただし後で説明するけど React Router は 10 年以上の歴史を持ちメジャーバージョンごとにドラスティックな変更を繰り返してきたため、バージョンアップを見送って昔のバージョンをそのまま使い続けている層がおそらくけっこういるはず。それが累積されているので npm trends のグラフだけでは最新のくわしい状況はよくわからない」

「React Router のライバルになりそうな存在はないんですか？」

「SPA 専用のルーティングライブラリとしては React Router v5 の改善・軽量版ともえる Wouter⁵ というプロダクトがあるけど、実績の少なさはもちろんアップデートの頻度も最近では減っていて将来性に疑問符がつく。あと知っておいて損はないのは TanStack Router⁶ かな。TanStack Query を始めとする数多くの有名 OSS プロダクトを生み出した Tanner Linsley 氏の手による TanStack コレクションのルーティングライブラリで、徹底した型安全性など開発者にニーズのある機能を他に先駆けて実装し、着実にユーザーを増やつつある。

ただ最近では React はフレームワークの下で使われるケースが多いので、それらのフレームワークが採用しているルーティングシステムも忘れちゃいけない。たとえば Next.js は独自のルーティングシステムを持っているから、それも比較検討の俎上に載せる必要がある」

⁵ <https://github.com/molefrog/wouter>

⁶ <https://tanstack.com/router>

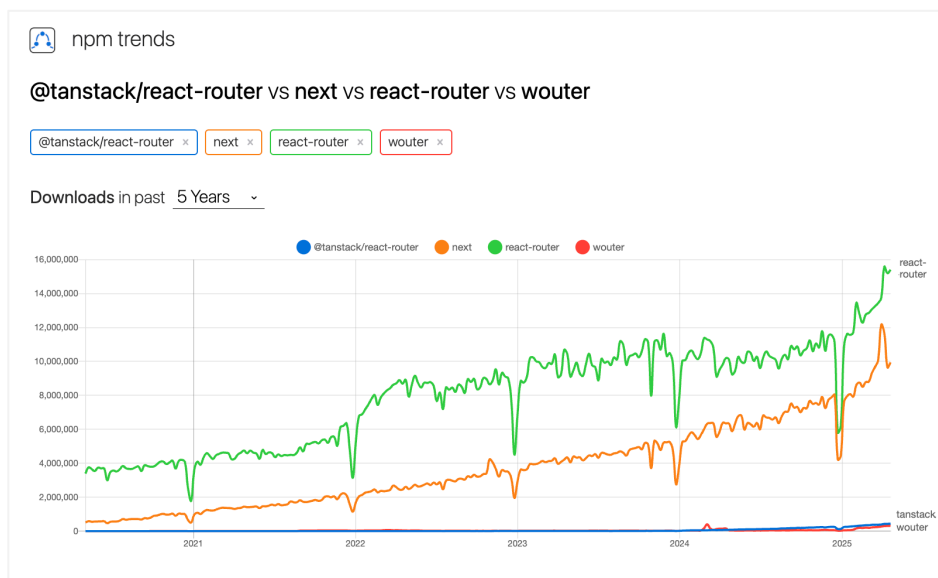


図 4: React 用ルーティングライブラリのトレンド (npm trends, May 2025)

「フレームワークとルーティングライブラリって、カテゴリーがちがうのに選定にあたってはいっしょに考える必要があるんですね」

「このルーティングシステムが使いたいから Next.js を採用する、といった人もめずらしくないからね。Remix チームが言っていたように、フレームワークの中身はそのかなりの部分がルーティングシステムで占められてるの。近年は TanStack Router をベースに TanStack Start というフレームワークが開発されたり、Remix v3 は React Router v7 として統合されたりと、その垣根がますます曖昧になってきてる」

「じゃあルーティングライブラリについては選定の余地はほぼなくて、フレームワークを選べばもれなくついてくるものと考えるべきものなんですか？」

「企業での新規プロジェクトなら純粋な SPA で運用するケースは激減してるから、そう言ってもいいだろうね。ただフレームワークが複数のルーティングシステムを持っている場合もあるから、採用したフレームワークによって一意に決まるわけじゃない。たとえば Next.js は最初の公開時からあった Pages Router と、2023 年 5 月リリースのバージョン 13.4 から導入している App Router の 2 種類のルーティングシステムを持ってる。そして両者は同じアプリケーション内で共存できる」

「うーん。React のルーティングシステムって、そんなにややこしいことになってるんですか？ いったいどれを選んだらいいんでしょう？」

「2025 年 5 月現在、商業レベルなら React Router か App Router (Next.js) の二択になるだろうね。Next.js の Pages Router は App Router より実績と安定性で上回るとはいえ、この先どれくらいメンテナンスやサポートを続けてくれるかが不明で、今から採用するにはリスクが高い。

結論としては、レアケースとは思うけど SPA で運用するなら React Router。サーバで稼働させる場合はアプリケーションの設計思想によって分かれて、データ取得および更新を RESTful にルーティングベースで行いたいならフレームワークモードの React Router。あくまでコンポーネントベースを貫いて Server Components をベースに組むなら Next.js の App Router ということになるかな」

「ふむふむ」

「というわけで、以降はその3つの使い方を学んでいくことにしよう」

11-3. React Router を使う

11-3-1. React Router の特徴

「React Router は Ryan Florence と Michael Jackson の2人が中心となって開発しているプロダクト。延べ10年以上の長い歴史を持っているけど、大きな破壊的変更や他プロダクトとの統合などが何度も起こっていて、そのたびにほぼ別のプロダクトとして生まれ変わっている。整理するために主なイベントを年表形式にまとめてみよう」

React Router 年表

2014 年 5 月：最初のバージョンを公開。

2016 年 10 月：バージョン 3.0 リリース。React 用のルーティングライブラリとしてデファクトスタンダードの地位を確立。

2017 年 3 月：バージョン 4.0 リリース。アーキテクチャを刷新し、後方互換性を完全に捨てたことで物議を醸す

2018 年 5 月：分家版の Reach Router⁷ を公開。積みもり積もった React Router の遺産を断ち切り、直感的でシンプルな設計とバンドルサイズの小ささによって急速に人気を集めていく。

2019 年 9 月：Hooks API を備えた React Router v5.1 をリリース。

2020 年 1 月：Reach Router を統合した React Router v6.0 の α 版をリリース（※実質は Reach Router v2）。

2021 年 4 月：フレームワークの Remix を公開。

⁷ <https://reach.tech/router/>

2022 年 9 月：Remix のデータフローをバックポートした Data API を備える React Router v6.4 をリリース。

2022 年 10 月：開発元の会社が Shopify に買収される。

2024 年 1 月：Remix v2.5 にて SPA モードを導入。

2024 年 12 月：Remix を統合した React Router v7.0 をリリース（※実質は Remix v3）。

「こうして並べてみると、まあ激動の歴史だよね。変化の激しいモダン Web 開発の世界で、公式なところともかくサードパーティのライブラリがトップシェアを維持し続けるには、常に変化し続ける必要があったんじゃないかな」

「開発者から不評を買ったメジャーバージョンアップというのはどれですか？」

「v3 → v4 と v5 → v6 のときだね。v3 ではコンポーネントの構造とは関係なく、ルート（root）の一箇所でルーティングの定義をする形式で、かつコンポーネントのライフサイクルに優先してルーティング発生時に任意の処理を行わせる機能を備えていた。それがコンポーネントベースの React の思想に反するという批判に応え、v4 ですべてをコンポーネントの下で行う設計に変更した。でも最新版の React Router では設定ファイルによるルーティングの集中管理と、コンポーネントツリーの外でデータ取得・更新を行う Route Loaders / Actions を備えてるのは皮肉な話。集中と分散の間を行ったり来たりしながら進化するコンピュータの歴史がここでも再現されてる」

「v5 → v6 のときは？」

「当時の React Router は歴史のしがらみによって複雑化した API と大きなバンドルサイズのため、初心者が手軽に扱えるものではなくなった。そこで Ryan Florence が、よりシンプルで直感的な API を備え、かつ当時ニーズが高まっていた Web アクセシビリティを重視した Reach Router を実験的に個人プロジェクトとして始めたの。それがたちまち人気を集め、一時はダウンロード数で React Router の 6 割近くまで迫った。しかし開発者が共通する 2 つのプロダクトがシェアを食い合う事態は彼らの望むところではなかったため、次期バージョンアップでそれらを統合することにしたのね。ブランドは React Router が残ったけど、設計や API は Reach Router からより多くを引き継いでいるため、v5 → v6 へは大々的な破壊的変更を伴うことになったわけ。同時にコードベースが TypeScript で書き直されたこともあり、ほぼ別物になったと言っていい」

「そういえば Remix v3 が React Router v7 に統合されたのって、そのときの再現みたいですね」

「まあ事情は似てるけど、今回の v6 → v7 のアップデートでは破壊的な変更はないよ。パッケージをアップグレードしてもアプリケーションはそのまま動作するはず。Shopify に買収されたあたりから、チームは下位互換性を大事にするスタイルに方向転換したように見える。まあ世界で 170 万以上の店舗と年間 400 億ドル以上の流通総額を持つ超巨大サイトで運用されるという重責を担うようになったので、慎重にならざるをえないんだろうけど」

「古くからのユーザーには遺恨を残してる人もいるけど、今は事情も変わって将来の互換性を心配す

る必要はないと」

「ただしそのために、React Router v7 では複数のルーティング定義形式が並立することになった。6.0 から存在するコンポーネントツリーによる形式、6.4 で導入された `createBrowserRouter` 関数を使う形式⁸、7.0 で導入された設定ファイル `routes.ts` に記述する形式、そして Remix から引き継いだファイルベースのルーティング⁹」

「4 つも！ どれを使ったらいいんですか？」

「前半の 2 つはライブラリ単体の環境でも使えるけど、後半の 2 つは Vite プラグインを導入する必要がある。また v7 の新機能がフルに使えるのは後半の 2 つのみ。設定ファイル形式は柔軟に定義できるけど、あらかじめ規則を決めておかないと後々カオスになりやすい。ファイルベースのほうはシンプルで直感的だけど、柔軟性では多少劣るのに加えて大文字・小文字の区別につけられない。また導入のために追加パッケージが必要になる」

「公式はどっちを推奨しているんですか？」

「どっちを推奨しているってことはないけど、公式ドキュメントは設定ファイル形式を前提に進められていて、ひな形作成ツールによるプロジェクトでもデフォルトで設定ファイル形式が適用されてるね。新しく出る機能も設定ファイル形式を前提にしていることもあるし、こちらにしておいたほうが無難と思う。この先はファイルベース形式はその命名ルールを参考にさせてもらう程度にして、設定ファイル形式でアプリケーションを作っていこう」

11-3-2. React Router のセットアップ

「ここまでの説明で事情はわかったと思うけど、React Router を使うには v6 系と同様のライブラリ単体での導入方法と、Remix が統合されたことによるフレームワークモードでの導入方法の 2 つがある。まずはライブラリ単体での導入のやり方を見ていこうか。まず SPA の React アプリケーションを作成し、そこに `react-router` パッケージをインストールする」

```
$ npm create vite rrlib-app --template=react-ts
$ cd rrlib-app/
$ npm add react-router
```

「さらにアプリケーションでルーティングライブラリの各種 API が使えるようにするために、コンポーネントツリーのトップレベルにプロバイダコンポーネントを設置する必要がある。通常は

⁸ <https://reactrouter.com/6.30.0/routers/create-browser-router>

⁹ <https://reactrouter.com/how-to/file-route-conventions>

`<BrowserRouter>` コンポーネント¹⁰を使い、その下に置いたルーティングコンポーネントの階層構造によってルーティングが定義される。実際に `src/main.tsx` を書き換えて試してみよう」

リスト 1: `BrowserRouter` によるルーティング設定

```
import { StrictMode } from 'react';
import { createRoot } from 'react-dom/client';
+ import { BrowserRouter, Navigate, Route, Routes } from 'react-router';
import './index.css';
import App from './App.tsx';

createRoot(document.getElementById('root')!).render(
  <StrictMode>
-   <App />
+   <BrowserRouter>
+     <Routes>
+       <Route path="/" element={<App />} />
+       <Route path="*" element={<Navigate to="/" replace={true} />} />
+     </Routes>
+   </BrowserRouter>
  </StrictMode>
);
```

「これはどういう挙動を意図してるんですか？」

「ルートパス (root path) `/` へのルーティングに `App` コンポーネントを設定し、それ以外のパスへのアクセスはルートパスにリダイレクトさせるというもの。変更前のアプリケーションではブラウザから `/foo/bar` など適当なパスにアクセスしてもそのまま表示されてたけど、変更後はそれが `/` にリダイレクトされるようになる」

「ほんとだ」

「この形式でトップレベルに置くプロバイダコンポーネントは `HashRouter` など他に何種類もあるけど、現在はこの `BrowserRouter` 以外はほぼ使う機会はない。そしてバージョン 6.4 から加わったのが `createBrowserRouter` 関数を使う方法。これによってさっきの定義をそのまま書き換えてみよう」

リスト 2: `createBrowserRouter` によるルーティング設定

```
import { StrictMode } from 'react';
import { createRoot } from 'react-dom/client';
import { RouterProvider, createBrowserRouter, Navigate } from 'react-router';
import './index.css';
import App from './App.tsx';
```

¹⁰ <https://reactrouter.com/6.30.0/router-components/browser-router>

```
const router = createBrowserRouter([
  {
    path: '/',
    element: <App />,
  },
  {
    path: '*',
    element: <Navigate to="/" replace />,
  },
]);

createRoot(document.getElementById('root')!).render(
  <StrictMode>
    <RouterProvider router={router} />
  </StrictMode>
);
```

「オブジェクト配列で設定するんですね。なぜ後になってこの形式が追加されたんでしょうか？」

「開発チームが Remix のデータフローを React Router にバックポートしようとした際、コンポーネントファイルに `loader()` などのあらかじめ名前が決められた関数を併記してエクスポートする形はそのまま流用できないので、ルーティング定義のところで設定できるようにしたの。たとえばコンポーネントに Router Loader をセットしたい場合は、データを返す任意の関数を値とした `loader` プロパティを `element` プロパティに併記する。`loader` や `action` のような **Data API**¹¹ が使えるように定義されたルータを **Data Router** と呼ぶこともある」

「じゃあ React Router で Route Loaders を定義したい場合に、この `createBrowserRouter` を使うわけですか？」

「いや、今ならその場合は React Router をライブラリ単体ではなくフレームワークモードでセットアップしちやったほうがいい。v7 の新機能の恩恵がフルに受けられるからね。従来型の SPA で運用したい場合は、SPA モードでビルドすればいい。ルートに対応するコンポーネントファイルに `clientLoader()` や `clientAction()` を置くと SPA でも Route Loaders / Actions が使えるよ」

「へー、そんなことができるんですね」

「じゃあ次は、React Router をフレームワークモードでセットアップしてみよう。既存のアプリケーションに後からマニュアルで導入することもできるけど、その場合は `react-router`、`@react-router/node`、`@react-router/serve`、`@react-router/dev` といったパッケージをインストールしたうえで複数の設定ファイルを編集し、さらにアプリケーションの構造も変える必要があるのでかなり面倒な作業

¹¹ <https://reactrouter.com/6.30.0/routers/picking-a-router#data-apis>

になる。新規にプロジェクトを始めるなら create-react-router¹² というひな形作成ためのコマンドツールが用意されているので、それを使うのがいちばん楽だね」

```
$ pnpm create react-router rrfw-app
```

「これによって構築されるプロジェクトのファイル構成は、アセットなどを除くと次のようになっているはず」

```
rrfw-app/  
  app/  
    routes/  
      home.tsx  
    welcome/  
      welcome.tsx  
    root.tsx  
    routes.ts  
    app.css  
  public/  
  react-router.config.ts  
  vite.config.ts  
  package.json  
  tsconfig.json
```

「アプリケーションコードを置くディレクトリは `src/` じゃなく `app/` なんですね」

「Remix の流儀をそのまま引き継いで。設定ファイルを編集して `src/` に変更することもできるけど、公式ドキュメントが参照しづらくなったり、変なところでバグを作りかねないのでこのままにしておこう。この生成されたアプリケーションがどのように動作するかを見ておきたい。まずチェックしておきたいのは Vite の設定ファイルである `vite.config.ts` だね」

リスト 3: create-react-router で生成した vite.config.ts

```
import { reactRouter } from "@react-router/dev/vite";  
import tailwindcss from "@tailwindcss/vite";  
import { defineConfig } from "vite";  
import tsconfigPaths from "vite-tsconfig-paths";  
  
export default defineConfig({
```

¹² <https://github.com/remix-run/react-router/tree/main/packages/create-react-router>

第 12 章 外部リソースのデータを取得する

12-1. 各種の外部データ取得手法

12-1-1. React における外部データ取得手法の変遷

「ここまではアプリケーションをシンプルなものにするため、必要なデータはプロジェクト内に置いた JSON ファイルをインポートする形で取得してた。でも実際の商業ベースのアプリケーションでは、データはネットワークを隔てた外部のリソースから取得するケースが多いよね。SPA ではほぼ 100% といっていいほど HTTP を介したサーバ API へのアクセスを用いていたけど、この手法はフレームワークによってロジックの実行場所がサーバに移ってもまだまだメジャー。だから次は React で外部データを取得する方法を学んでいこう」

「HTTP リクエストによるデータ取得って `fetch()` を使うだけですよ。そんなに仰々しく学ぶ必要があるものなんですか？」

「取得するだけならそう難しくないけど、コンポーネントツリーのどこで取得・管理するかとかレンダリングとの兼ね合いをどうするかといった問題があってけっこう複雑なのよ。それゆえ React がリリースされてから今まで新しいものが現れては消えて、主流の手法が何度も入れ替わってきた。ベテランならデータ取得まわりの記法を見るだけで、これはいつごろ書かれたコードかというのがわかるくらい」

「ええー、そんなプロファイリングができるほどですか？ Rails では考えられませんが、React ではなぜデータ取得くらいでそんなに変遷が激しいんでしょうか？」

「SPA におけるデータ取得のベストプラクティスが確立されてなかったからね。だからもっともユーザー数の多い React でさまざまな手法が提案され、そこでより多くの賛同者を集めた手法が既存のものを駆逐するという形で進化していったわけ。そして今度はサーバサイドを包含したメタフレームワークが台頭するようになり、それに合わせた手法が提案されて競い合ってる状況」

「なるほど」

「順を追って説明していこう。2013 年の React 公開後、最初に主流になったのはクラスコンポーネントの **ライフサイクルメソッド** を用いる方法だった。リリース当初はコンポーネントはクラスで記述されていたこと、そのライフサイクルに結びついたメソッドがあったという話は以前にしたよね⁷²。まず取得すべきデータの持ち回り先をコンポーネントの `state` として定義し、初期値を入れておく。初期値の場合は UI にスピナーやスケルトンを表示するようにしていると、初回レンダリングのとき

⁷² 【② React 基礎編】の「8-5-1. はじめはクラスだったコンポーネント」を参照のこと。

にはそれが適用される。そしてライフサイクルメソッドの `componentDidMount()`⁷³ を定義してそこに非同期のデータ取得のロジックを記述しておけば、最初のレンダリング直後にその処理が走ってデータの取得が終わり次第に state が書き換えられる。すると React の Reconciler がそれを検知して新しい state 値を適用した再レンダリングが行われる、というのが一連のプロセスだね」

「ふーむ、データ取得とコンポーネントのレンダリングの兼ね合いが難しいというのはそういうことなんですね。外部 API との通信には時間がかかるから、そこで UI の表示を止めない工夫が求められると。Rails のようなサーバサイドフレームワークではデータ取得も同期処理だから、クリックしてから次のページ遷移まで一瞬待たされるのは仕方ないとあきらめてしまうところですけど、SPA は作り込めば UX の向上をいくらでも追求できますもんね」

「しかしこのやり方にはロジックが個別のコンポーネントの状態やクラス要素と密結合になりがちで再利用性に欠けてる、同じデータを複数のコンポーネントで重複して取得してしまうといった課題があったの。そこで注目されたのが **Redux**⁷⁴。コンポーネントから独立したグローバルな状態を扱うためのライブラリで、2015 年に登場するやあっという間にデファクトスタンダードになった。Redux ではグローバルな状態を単一かつ読み取り専用のものとして持ち、その変更手続きを厳格な純粋関数で定義したものに限定する。Redux は単体では副作用を扱うのは難しかったんだけど、Redux Thunk⁷⁵ や Redux-Saga⁷⁶ といったミドルウェアを組み合わせることで状態の変更手続きに非同期のデータ取得処理を差し込むことが可能になる。この手法だとデータの重複が避けられるし、ロジックの再利用もやりやすくなる。だから当時は React による開発では必ずと言っていいほど Redux とそのミドルウェアがセットで導入されてたの」

「React 初心者の私でも聞いたことがあるほど、Redux ってかなりメジャーなライブラリでしたよね。でも問題はなかったんですか？」

「けっこうあったよ。まず概念が難解で学習コストが高いこと。やりたいことはただのデータ取得なのに実現するための手段がやたら回りくどく、書かなければいけないコード量がものすごく多い。当時はそれが高尚に感じられてみんな受け入れてたけど、今となってはなぜあんなに普及していたかは不思議だね。Redux がわからなくて React を学ぶのを挫折した人も当時は少なくなかったはず」

「その時期に React を始めなくてよかった……。でもなぜそんなに勢いがあった Redux が使われなくなったんですか？」

「2019 年の React 16.8 で Hooks API が導入されたことが大きい。それにより関数コンポーネントで副作用を扱えるようになり、しかもその記述様式が非常に簡潔で明快だった。また必要に応じて

⁷³ <https://ja.legacy.reactjs.org/docs/react-component.html#componentdidmount>

⁷⁴ <https://redux.js.org/>

⁷⁵ <https://github.com/reduxjs/redux-thunk>

⁷⁶ <https://redux-saga.js.org/>

Custom Hooks⁷⁷ として依存する state 変数ごとコンポーネントから切り出せるため、再利用性も担保される。データ取得処理を Hooks を使って書くと、Redux ミドルウェアを使っていたときよりも非常にシンプルでわかりやすく書けるので、みんな Redux の熱狂から覚めていった。ところでこの動きを俯瞰して見ると、データ取得の分野でも集中と分散を行きつ戻りつして技術が発展してることがわかるよね]

「ライフサイクルメソッド（分散）→ Redux ミドルウェア（集中）→ Hooks（分散）というわけですね。でも集中管理なら通信の重複を避けるのは容易だったと思いますが、また分散管理に戻ってきたときにそこはどうしたんですか？」

「いい質問だね。その懸念は適切で、Hooks API だけではフォローできない問題だった。そのニーズに応えるべく登場したのが **React Query**⁷⁸ や **SWR**⁷⁹ といった Hooks 時代の新しいデータフェッチライブラリ。リクエストにユニークなキーを割り振り、メモリキャッシュと関連付けて管理できるようになっていた。加えてローディングやエラー状態の管理もシンプルなインターフェースで行えるとあって、2020 年ごろから急速に普及していった」

「ここでキャッシュが出てくるんですね。でもシンプルに書いてキャッシュも効くとなると、これ以上はもう改良の余地がなさそうに思えますけど」

「いや、React コアチームはそこにまだいくつも課題があると考えてた。ひとつはコンポーネントツリーに必要以上の再レンダリングが起きてしまい、体感パフォーマンスの悪化につながる。もうひとつはローディングやエラー状態のハンドリングが条件分岐を用いた手続き型の記述になってしまったこと。前者はかなり根が深い問題で、従来の React ではコンポーネントツリーのレンダリングは Blocking Rendering といって、一度始まると中断も割り込みもできない同期的なものだった。コアチームは数年スパンでこの改修に取り組み、まずコンポーネントの更新処理を細かいタスクに分割し、優先度に基づいて処理をスケジューリングできるようにした非同期レンダリングエンジンの **Fiber**⁸⁰ を React 16.0 で導入。そこから慎重に開発を進め、各種処理を中断・再開可能にした **Concurrent Rendering** を 2022 年 3 月の React 18.0 で安定版の機能としてリリースした⁸¹。これに含まれる **Suspense** という機能が、データの取得・表示を行うのに非常に便利だったの」

⁷⁷ 【② React 基礎編】の「8-6. Custom Hooks でロジックを分離・再利用する」を参照のこと。

⁷⁸ Tanner Linsley の手による、キャッシュ機構を備えたデータフェッチライブラリ。彼は「TanStack」を冠する数多くのライブラリやフレームワークを開発している著名な専門の OSS 開発者であり、React Query も v4 リリース時に TanStack Query にリブランドされた。
<https://tanstack.com/query>

⁷⁹ Vercel が開発している、キャッシュ機構を備えたデータフェッチライブラリ。2024 年以降はプロジェクトが停滞気味だが、SWR で得た知見が Next.js のキャッシュ機構の設計・開発に大いに活用されている。
<https://swr.vercel.app/ja>

⁸⁰ <https://github.com/acdlite/react-fiber-architecture>

⁸¹ 「React v18.0 – React」<https://ja.react.dev/blog/2022/03/29/react-v18>

「それがあればコンポーネントの state や副作用を使うことなく、データ取得ができるようになるんですか？」

「そう。JavaScript では throw できる値に制限がないという話は前にもしたよね。Suspense はそれを利用したしくみになっていて、throw された未解決の Promise を捕捉してそれが解決するまでレンダリングを中断してくれるというものなのね。そしてコードの記述も宣言的かつシンプルになる」

「じゃあその Suspense を使えば、さっき挙げられた React Query でしたっけ？ あれらのライブラリはもう必要なくなるということでしょうか？」

「いや、React Query も SWR もかなり早い段階で Suspense をサポートしてた。というのも React コアチームがサードパーティの開発者向けに実験的機能として Suspense を公開してたのね。むしろ Suspense をデータ取得に利用するのにこれらのライブラリを使わなければいけない期間が4年近くも続き、本体のみでそれを行うには 19.0 でリリースされた use() という Hooks API の登場を待つ必要があった」

「うーん、複雑ですね……。手っ取り早く今のデータ取得におけるベストプラクティスを教えてください！」

「そうだね。現状での SPA におけるベストプラクティスは、React Query 改め TanStack Query もしくは SWR と Suspense の併用かな」

「あ、サーバではまた話が違ってくるんですね」

「SSR とデータ取得は基本的にあまり相性がよくない。通常はデータ取得の完了を待ってからページを HTML にレンダリングしてクライアントに返すことになる。だから従来 Next.js などでは初回アクセスのページにデータ取得処理が含まれていると、Ruby on Rails などのサーバサイドフレームワークと同様のもっさりした表示になってしまっていた。Suspense は React 18 のリリース当初から SSR と協調して動作する設計になっていて、この問題を解決してくれる」

「Suspense ってサーバサイドでも動作するんですか？」

「React 18 では SSR が Suspense と協調して動作するよう、2つの機能が提供された⁸²。ひとつはサーバから HTML を一度のレスポンスで返すのではなく、チャンクに分割してストリーミングする **Streaming HTML**。もうひとつはページ全体を一度に Hydrate するのではなく、必要な部分だけを必要なタイミングで Hydrate する **Selective Hydration**。

これらを活用した場合の挙動は次のようになる。サーバにリクエストがあると、データ取得の開始と同時にページ全体を HTML にレンダリングしたうえでストリーミング形式でクライアントに送信する。クライアントでページが表示されその部分の Hydration が終わっても、サーバとは繋ぎっぱなしの状態が続く。そしてサーバでデータ取得が完了すると、その部分を HTML にレンダリングしてチャンクにしてクライアントに送信。クライアントはその部分の HTML を差し替え、さらに必要で

⁸² 「New Suspense SSR Architecture in React 18 · reactwg/react-18 · Discussion #37」
<https://github.com/reactwg/react-18/discussions/37>

あればその Hydration も行う」

「だいぶ複雑なことをしてるようですが、なんだかスマートで速そうということは伝わります」

「Google が Web サイトのパフォーマンスを図るために **Core Web Vitals** という指標を提唱して、その中にユーザーがアクセスしてからページのコンテンツのいずれかが画面にレンダリングされるまでの時間を意味する FCP (First Contentful Paint) ⁸³と、アクセスしてから操作可能になるまでの時間を意味する TTI (Time To Interactive) ⁸⁴というものがある。Streaming HTML は FCP を、Selective Hydration は TTI を向上させるための技術と言えるね。これらによってデータ取得処理を含んだページの SSR の体感パフォーマンスがかなり向上する」

「ということは現状、Suspense の利用は SPA においても SSR で運用する場合においてもベストプラクティスになってるわけですか」

「そう言っても差し支えないと思う。説明が長くなったのでデータ取得手法の変遷を、年表形式であらためて振り返ってみたい」

- クラスコンポーネントの `state` と `componentDidMount()` を用いる (2013 年～2015 年頃)
- Redux ミドルウェアの利用 (2015 年～2019 年頃)
- 関数コンポーネントの `state` と `useEffect()` を用いる (2019 年頃から)
- React Query や SWR の利用 (2020 年頃から)
- クライアントでの Suspense の利用 (2020 年頃から)
- SSR with Suspense (2023 年頃から)

「今回は勉強のために、Suspense を使わない場合と使った場合の両方の書き方を学んでおこう。なおそれに際して React Query や SWR などのサードパーティーのライブラリは使わないでおく。そのうえで SPA と Next.js の App Router、それに React Router v7 のフレームワークモードの 3 つのパターンを紹介する」

⁸³ <https://web.dev/articles/fcp?hl=ja>

⁸⁴ <https://web.dev/articles/tti?hl=ja>

12-1-2. アプローチからデータ取得手法を分類する

「React における各種データ取得の手法を出現した順に時系列に沿って紹介したけど、これらの手法はコンポーネントのレンダリングとの兼ね合いの観点から 3 つのアプローチに分類して説明することもできる⁸⁵」

- **Fetch-on-Render** (レンダリングした上でフェッチ)
..... 最初のコンポーネントツリーのレンダリング完了後にデータ取得を開始し、取得後に再レンダリングする
- **Fetch-then-Render** (フェッチしてからレンダリング)
..... 先にデータを一括取得して、その完了後にレンダリングを行う
- **Render-as-You-Fetch** (フェッチしながら逐次レンダリング)
..... そのコンポーネントのレンダリングをピンポイントで中断してデータ取得を始め、取得後にレンダリングを再開する

「さっき挙げてもらったもののうち、どれがどれに該当するんですか？」

「さっきは主にこういった API やライブラリを使うかという観点からカテゴライズしたので、きっちり一対一で当てはめるのは難しいね。でも強いて言うなら Suspense を使う場合は Render-as-You-Fetch、Suspense を使わずコンポーネントツリーの上部やルーティング実行時に一括してフェッチする場合は Fetch-then-Render、それ以外が Fetch-on-Render になるかな」

「それぞれデメリットはありますか？」

「Fetch-on-Render はデータ取得が終わらない内はローディング UI などを表示できるけど、データ取得開始のタイミングが最初のレンダリングが終わった後になってしまう。またデータ取得を行うコンポーネントが入れ子になっているといわゆる **Network Waterfall**⁸⁶ が起きやすい。

Fetch-then-Render アプローチではそれが解消される。必要なデータをあらかじめ一括して取得しておき、それをツリーの上部からコンポーネントに流し込む。そのしくみ上 SSR のフレームワークで用いられることが多く、React Router の Route Loaders や、Next.js の Pages Router の `getServerSideProps()` をそのまま使った場合はこの Fetch-then-Render になる。ただしこれらはデータ取得がコンポーネントのレンダリングをブロックしてしまう。Core Web Vitals に即して言うと、ページの主要コンテンツが表示されるまでの時間である LCP (Largest Contentful Paint)⁸⁷ は Fetch-on-

⁸⁵ 「Suspense for Data Fetching (Experimental) – React」

<https://17.reactjs.org/docs/concurrent-mode-suspense.html#traditional-approaches-vs-suspense>

⁸⁶ 複数のリクエストが順次実行されることで、全体の処理時間が長くなる問題のこと。各リクエストが前のリクエストの完了を待ってから次のリクエストを開始するため、その積み重なりで直線的にパフォーマンスが悪化する。

⁸⁷ <https://web.dev/articles/lcp?hl=ja>

Render よりも有利だけでも、FCP はどうしても長くなってしまう」

「つまりクリックしてからページ遷移まで固まってるが長いと」

「そうだね。そしてここまで出てきた各種の問題を解消するべく編み出されたのが Render-as-You-Fetch アプローチ。レンダリング中に未解決の Promise に包まれたデータに行き当たったら、そのコンポーネントのレンダリングを中断して用意された fallback（縮退運用のための UI）を表示しておき、Promise 解決後にそのデータでレンダリングを再開するというもの。Fetch-on-Render のようにツリー全体のレンダリングを待つことなくコンポーネント評価時にデータ取得が開始される、また Fetch-then-Render のようにレンダリングをブロックすることなく取得が完了したデータから順次表示されていく」

「おおー、まさにいいとこ取りのアプローチなわけですね」

「ただしコンポーネントの設計が悪いと Network Waterfall の問題は変わらず残るし、A と C のデータ取得が終わっているのに B のデータ取得が終わってないからコンテンツの表示がまとめてブロックされてしまうような事態も起きうる。React の思想では個々のコンポーネントが必要なデータを自律的に取得して、錯綜する通信の最適化はシステムに丸投げで OK というものだったけど、実際には複数のデータ取得を考慮したコンポーネント設計がどうしても必要になってくるわけね」

「まあ、そうそう理想通りにはいきませんよね……」

「今から同じアプリケーションを、これら 3 つのアプローチでそれぞれ作ってみよう。チームの考え方や採用しているフレームワーク、さらには開発した時期などによってどのアプローチが採用されているかはまちまちだけど、秋谷さんにはどんなプロジェクトを担当することになってもらいたいかな」

12-2. Fetch-on-Render アプローチ

「題材には『スラムダンク』のチーム別メンバー表示アプリをベースに改変したものを使う。全体の基本的な構造はすでにわかっているはずだからね。あのアプリではメンバーのデータをローカルの JSON ファイルから読み込んでいたけど、今回はそれを外部のサーバ API から取得する形にする。具体的には、GitHub REST API にアクセスして任意の組織の開発メンバーの情報をリストで表示するというアプリになる。まずは実行画面を見てもらおうかな」

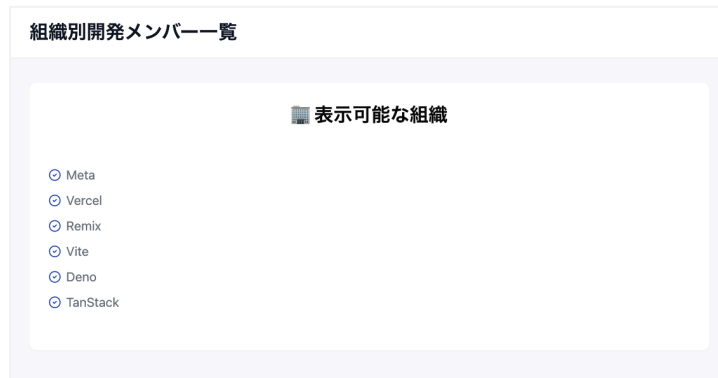


図 9: 開発メンバー表示アプリのトップページ

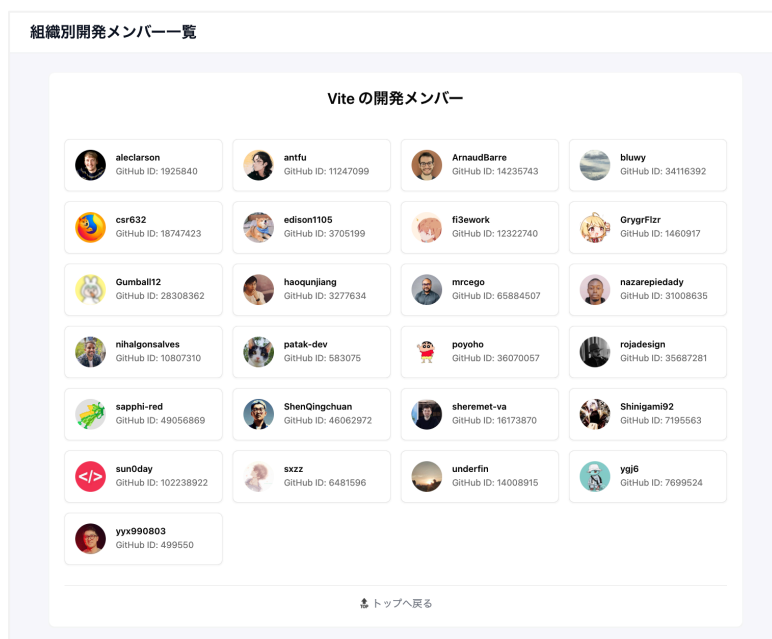


図 10: 同メンバー一覧ページ

「たしかにアプリケーションの構造はほぼ同じですね。メンバー一覧ページのところで GitHub API からデータを取得してるわけですか」

「そうだね。ではまず最初にコンポーネントの副作用処理の中でデータ取得を行う Fetch-on-Render

アプローチを用いた書き方から学んでいこう。用意したのはライブラリモードの React Router によってルーティングを行う SPA のプロジェクト⁸⁸で、ページの構成は以下の 2 つ」

- `/.....` 組織一覧を表示するトップページ
- `/:orgId/members .....` Organization ID が `orgId` の組織の開発メンバー一覧ページ

「ちなみに GitHub API へのアクセスの部分はどのアプローチでも共通になる。本題に入る前にその部分をチェックしておこう」

リスト 32: `src/entities/data-reader.ts`

```
import camelcaseKeys from "camelcase-keys";
import type { Member, Organization } from "@entities/types.ts";
import { isMember, isObject } from "../type-validator.ts";
import {
  :
}
export async function getMembers(orgId: string): Promise<Member[]> {
  const resource = `https://api.github.com/orgs/${orgId}/members`;

  const response = await fetch(resource, {
    headers: {
      Accept: "application/vnd.github.v3+json",
      "User-Agent": "Node.js",
    },
  });

  if (!response.ok) {
    throw new Error(`HTTP error status: ${response.status}`);
  }

  const data = (await response.json()) as unknown;

  if (!Array.isArray(data)) {
    throw new Error("Invalid API schema");
  }

  const members = data.map((obj) => {
    if (!isObject(obj)) throw new Error("Invalid API schema");

    const member = camelcaseKeys(obj, { deep: true });
  });
}
```

⁸⁸ <https://github.com/klemiary/Riakuto-StartingReact-ja5.0/tree/main/12-data/02-on-render>

```

    if (!isMember(member)) throw new Error("Invalid API schema");

    return member;
  });

  return members;
}

```

「利用するのは GitHub API のバージョン 2022-11-28 の Organization メンバーのエンドポイント。なお認証なしでの利用は同一の IP アドレスで 1 時間に 60 回までのレート制限があるので気をつけてね。API のくわしい仕様は公式ドキュメント⁸⁹にあるけど、端的には

`https://api.github.com/orgs/{orgId}/members` にアクセスすると `orgId` の組織のメンバーリストが JSON データの形で返ってくるということ」

「でもこの `getMembers()`、ただ API をフェッチするだけじゃなく、他にも何やら色々やってますよね」

「そう見えるのは、主に型の整合性を厳密にチェックしてるからだね。`fetch()` は戻り値が `Promise<any>` なので、そのまま扱うと随所で型にまつわるエラーが起きてしまう。この関数で最終的に返したいのは `Member` オブジェクトの配列。だからまずは `unknown` として受け取り、次にそれが配列かどうかを検証。そして配列の中身が `Record<string, unknown>` のオブジェクトかどうかを検証、最後にそれが `Member` インターフェースに適合しているかどうかを検証してる」

「`camelcase-keys` というライブラリをインポートして使ってるのは？」

「GitHub API が返す JSON データのプロパティ名が JavaScript で一般的なキャメルケースではなく、`avatar_url` のようなスネークケースになっているため、ロジック内で扱うのに不都合なのね。`camelcase-keys` はその名のとおり、オブジェクトのキーをキャメルケースにしてくれるライブラリ⁹⁰。ただ引数が `Record<string, unknown>` のオブジェクトまたはその配列でないと受け付けてくれないため、その前にユーザー定義の型ガードである `isObject()` 関数で型を検証してる」

「そしてその下で使ってる `isMember()` は同じく `Member` オブジェクトかどうかを検証するユーザー定義の型ガード関数というわけですか。合わせて 3 回も型の検証をする必要があるの、けっこう面倒ですね」

「外部データを扱うには、それだけ慎重にならないといけないわけだよ。実行時エラーは起こしたく

⁸⁹ 「Organization メンバーの REST API エンドポイント - GitHub Docs」
<https://docs.github.com/ja/rest/orgs/members?apiVersion=2022-11-28>

⁹⁰ <https://github.com/sindresorhus/camelcase-keys>

ないでしょ。じゃあ次はいよいよ本丸に攻め込もう。この `getMembers()` を使って取得したメンバーリストを表示するためのコンポーネントね]

リスト 33: src/components/MemberList.tsx

```
import { useEffect, useState } from "react";
import { Loader2 } from "lucide-react";
import type { Member } from "@entities/types.ts";
import { getMembers } from "@entities/data-reader.ts";
import MemberCard from "../MemberCard.tsx";

interface MemberListProps {
  orgId: string;
}

export default function MemberList({ orgId }: MemberListProps) {
  const [members, setMembers] = useState<Member[]>([]);
  const [isLoading, setIsLoading] = useState(false);

  useEffect(() => {
    async function fetchMembers() {
      setIsLoading(true);

      try {
        const membersData = await getMembers(orgId);
        setMembers(membersData);
      } catch (error) {
        if (error instanceof Error) {
          console.error(error.message);
          throw new Error("GitHub API Error", { cause: error });
        }
      } finally {
        setIsLoading(false);
      }
    }

    void fetchMembers();
  }, [orgId]);

  return isLoading ? (
    <div className="my-14 flex h-80 items-center justify-center">
      <Loader2 className="size-12 animate-spin text-primary" />
    </div>
  ) : (
    <div className="mt-10 grid grid-cols-1 gap-4 sm:grid-cols-2 md:grid-cols-3 lg:grid-cols-4">
```



```

    {members.map((member) => (
      <MemberCard member={member} key={member.id} />
    ))}
  </div>
);
}

```

「あ、私に解説させてくださいね。まず state 変数 `members` でメンバーリストのデータを持ち回るようにしてる。同じく state 変数 `isLoading` によってデータのローディング状態を扱う。そして `isLoading` が `true` のときはローディング UI を表示し、`false` の場合は配列内のメンバーのデータを `<MemberCard>` コンポーネントに渡して表示させてると。ここまではわかりますけど、副作用のロジックの中で何をやってるかが難しいですね。いきなり関数を定義してるのはどういうことでしょうか？」

「`useEffect` に渡すセットアップ関数は非同期関数にできないので、そのままでは `getMembers()` を呼ぶことができない。だから中で戻り値のない非同期関数 `fetchMembers` を定義して、あらためてそれを実行してるわけ。そのコール時に `void` をつけてるのは、戻り値の `Promise` を明示的に無視することを示してる。

`fetchMembers()` はまず `isLoading` を `true` にし、`try...catch` 文の中で `getMembers()` をコールして結果を `members` に格納する。`finally` ブロックの中で `isLoading` を `false` にしてコール後に必ずそれが実行されるようにする」

「なるほど。そしてこの処理は依存配列に `orgId` が設定されているため、初回および `orgId` が変更されるたびに実行されるというわけですね。ところで基本的な疑問なんですけど、これは Fetch-on-Render アプローチに分類されるということでしたよね？ レンダリングした上でフェッチするってことでしたけど、副作用処理はそのコンポーネントが評価されるときに実行されるんじゃないんですか？」

「React ではコンポーネントの副作用ロジックは評価時には実行されず Fiber が確保している領域のキューに登録されて、ツリー全体のレンダリングが終わった直後にまとめて実行されるの」

「えっ、知らなかった……」

「だからここでやっているのはまず問答無用にローディング UI を表示しておき、あとで実行される副作用の中でデータを取得し、その値で state を書き換えて再レンダリングを発生させてるわけね」

「よくわかりました。ところでさっき、このローディング UI って表示されましたっけ？ 一瞬だったのを見逃してしまいました」

「じゃあ直接 `isLoading` を書き換えて強制的にローディング UI を表示させよう。ブラウザ拡張の React Developer Tools は以前インストールしたよね？ それを開いて Components タブを表示。コンポーネントのリストから `MemberList` を選択すると右ペインに props や hooks の一覧が表示される。hooks の 2 番目の State が `isLoading` なので、その値の `false` を `true` に書き換える。どれが `isLoading` かわかりにくければ、右上の  アイコンをクリックすると変数名が表示されるよ」

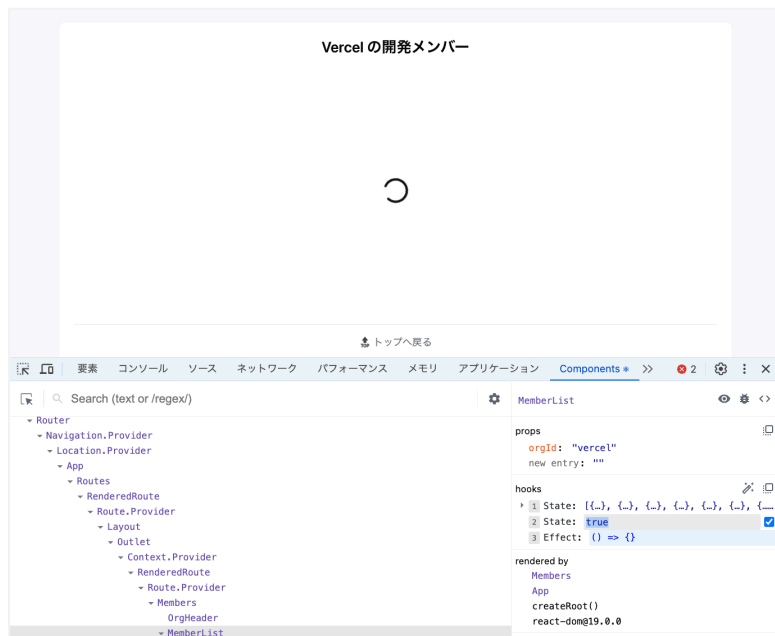


図 11: 開発メンバー表示アプリのトップページ

「あっ、ぐるぐるが表示されました！　へー、React Developer Tools って props や state を直に書き換えることもできるんですね」

「便利でしょ。members の中身も  をポチポチ開いていけば確認できるよ」

「ほんとだ、これは見やすいですね。console.log() を記述してダンプするよりお手軽だし。……あれ？　今ついでに『ネットワーク』タブも確認してみたんですけど、なんか GitHub API へのリクエストが 2 回ずつ発生してませんか？」

「ああこれはね、**Strict Mode** を有効にしている際に開発環境でのみ起こる挙動なの。コンポーネントが純粋関数であれば何度実行しても結果は変わらないはずなので、わざと 2 回呼び出すことでバグに気づきやすくする。公式ドキュメントには『開発中の二重レンダリングによって見つかったバグの修正』や『開発中にエフェクトを再実行して見つかったバグの修正』といった具体的なケースが挙げられているので、時間があるときに読んでおくといいよ⁹¹。create-vite を使って生成したプロジェクトでは src/main.tsx で `<StrictMode>` がコンポーネントツリーの最上位にマウントされるようになってる。もちろんビルドして走らせた場合にはこの挙動は起きないので心配しなくていいよ」

「なるほど、わかりました」

⁹¹ 「`<StrictMode>` – React」 <https://ja.react.dev/reference/react/StrictMode>

第 13 章 エラーをハンドリングする

13-1. Error Boundary とは

「ここまで特に深く考えず気軽にエラーを throw していたけど、それが適切に catch しきれなかった場合に React アプリケーションはどうなるか。ページが真っ白になって、ブラウザのコンソールには次のような捕捉されなかったエラーが表示される」



図 13: ブラウザコンソールに表示されたエラー

「ちゃんと `try...catch` 文を使ってないから……」

「まあそうなんだけど、React は宣言的なプログラミング作法に基づいてアプリケーションが構築されるので、命令的な `try...catch` 構文をエラーが発生しそうなところに逐一書くのにそもそも向いてないんだよ。とりあえず、この警告メッセージをちゃんと読んでみよう」

Consider adding an error boundary to your tree to customize error handling behavior.
Visit <https://react.dev/link/error-boundaries> to learn more about error boundaries.

「なにになに、エラーハンドリングの挙動をカスタマイズするために error boundary を加えることを検討してください……。この **Error Boundary** っていうのは何ですか？」

「その配下の子孫コンポーネントで発生したエラーを一括して捕捉し、任意の処理や UI 表示をさせるための特別なコンポーネントのこと。Boundary は『境界』のことなので、中で発生したエラーを外

に漏れ出ないようにするための境界という意味になるかな。React 16 で取り入れられた機能で¹¹⁸、これを使えばコンポーネントツリーの中で宣言的にエラーをハンドリングできるようになる」

「ふーん。じゃあ React 本体がその Error Boundary のコンポーネントを提供してくれてるんですか？」

「残念ながらそうじゃない。React 16 以降で提供されているのは、クラスコンポーネントで子孫コンポーネントのレンダリング中に発生したエラーがあった場合に呼び出されるライフサイクルメソッドの `componentDidCatch()`¹¹⁹。そして同じくエラーが発生した際に直接コンポーネントの state を変更して再レンダリングをトリガーできるようにした静的メソッドの `getDerivedStateFromError()`¹²⁰。そして後者がリリースされた 2018 年 10 月以降、今に至るまで一切機能がアップデートされてない。React 本体のみで Error Boundary を構築するには、これらのクラスコンポーネントのメソッドを使って自作するしかない。以下は公式ドキュメントに掲載されている作例ね¹²¹」

```
class ErrorBoundary extends React.Component {
  constructor(props) {
    super(props);
    this.state = { hasError: false };
  }

  static getDerivedStateFromError(error) {
    return { hasError: true };
  }

  componentDidCatch(error, info) {
    logErrorToMyService(error, info.componentStack);
  }

  render() {
    if (this.state.hasError) {
      return this.props.fallback;
    }

    return this.props.children;
  }
}
```

¹¹⁸ 「Error Handling in React 16 – React Blog」

<https://legacy.reactjs.org/blog/2017/07/26/error-handling-in-react-16.html>

¹¹⁹ 「Component – React」 > 「`componentDidCatch(error, info)`」

<https://ja.react.dev/reference/react/Component#componentdideatch>

¹²⁰ 「Component – React」 > 「`componentDidCatch(error, info)`」

<https://ja.react.dev/reference/react/Component#static-getderivedstatefromerror>

¹²¹ 「Component – React」 > 「エラーバウンダリでレンダー中のエラーをキャッチする」

<https://ja.react.dev/reference/react/Component#catching-rendering-errors-with-an-error-boundary>

```
}
}
```

「ということは、Error Boundary を実装するためだけに今さらクラスコンポーネントでこういうのを自作する必要があるということですか？」

「公式も最初は関数コンポーネントで使えるものをそのうちリリースすると言ってたんだけどね。ちなみに Preact にはそれに対応する `useErrorBoundary()` という Hooks API が存在してる¹²²。技術的に目新しいトピックではないから担当したがる人がいないのか、新しいドキュメントではもう開き直って `react-error-boundary`¹²³ のようなライブラリを使えと書いてあったりする」

「それって Meta が作ってるライブラリなんですか？」

「いや作者は React コアチームの人だけど、個人のプロダクトとして開発されてる。使用実績は十分にあるし、アップデートもずっと継続されてるので直近での使用はまあ問題なさそう。長期的には保証できないけど、そもそも Meta が会社として出したものでも今は開発が止まってしまったものがめずらしくないので、それは同じこと」

「たしかに」

「また最近ルーティングライブラリやフレームワークが Error Boundary のための API を提供してくれていることが多いので、プロジェクトの環境に応じてそれらの採用を積極的に検討するべきだね」

「Next.js や React Router にもその機能はあるんですか？」

「うん。Next.js の App Router、React Router では `createBrowserRouter()` を使った Data Router およびフレームワークモードで使える API が提供されてる。だから今回も SPA で `react-error-boundary` を使う場合、Next.js の App Router、React Router v7 のフレームワークモードの 3 パターンでエラーをハンドリングしてみよう」

13-2. react-error-boundary を使う

「一般的に Web アプリケーションでエラーとして対応すべき主なパターンは次の 2 つ」

1. アクセスしようとしたページは存在しないという **Not Found**
2. それ以外で何かしらシステムに障害が起きた場合

¹²² 「Hooks – Preact Guide」 > 「`useErrorBoundary`」
<https://preactjs.com/guide/v10/hooks/#useerrorboundary>

¹²³ <https://github.com/bvaughn/react-error-boundary>

「そういえば Next.js の App Router でも、ファイル名の規約として用意されていたのは `not-found.tsx` と `error.tsx` でしたね」

「長年の知見からそこに行き着いたんだろうね。さて今回サンプルに用いるのは GitHub REST API から組織別の開発メンバーを取得して表示するアプリケーション。従来型の SPA としては SWR を使って `Suspense` で表示するバリエーションを改変しよう。既存の挙動として、アクセスしたパス `/:orgId/members` の `orgId` が用意されたセットのどれにも適合しない場合はトップにリダイレクトしてた。それを次のような UI が表示されるように変えたい」



図 14: 意図的なエラーを発生させて表示したエラー画面

「前述のパターン 2 のエラーは狙って生じさせるのが難しいので、`/error/members` というパスにアクセスしたときにわざとエラーを `throw` するという変則的な仕様にするよ。プロジェクトが用意してあるので¹²⁴、まずはその記述を入れた `src/components/OrgHeader.tsx` の変更部分を見てみよう」

リスト 47: `src/components/OrgHeader.tsx` の差分

```
⋮
export default function OrgHeader({ orgId }: OrgHeaderProps) {
+   if (orgId === "error") {
+     throw new Error(`Intentional Error!`);
+   }
  const org = getOrganization(orgId);

  if (!org) {
-    return <Navigate to="/" replace />;
+    throw new Error(`Not found organization '${orgId}'`, { cause: 404 });
  }
}
```

¹²⁴ <https://github.com/klewiary/Riakuto-StartingReact-ja5.0/tree/main/13-error/02-reb>

```

    return (
      <>
        <title>`${org.name}の開発メンバー`</title>
        <h2 className="mb-12 text-center">{org.name} の開発メンバー</h2>
      </>
    );
  }

```

「new Error() の第2引数に渡してる cause って何でしたっけ？」

「以前は Error オブジェクトのコンストラクタには引数はエラーメッセージのひとつしか渡せなかったんだけど、ES2022 でエラーの原因を特定するための cause プロパティを備えたオブジェクトがオプションとして渡せるようになった。一般的には try 構文の catch ブロックで別のエラーを throw する際に、捕捉したエラーをこの値に設定することでスタックトレースを失わせないようにする使い方が多い。でも今回はエラーステータスの数値を入れて、Error Boundary 側で処理の振り分けをするようにしたの」

「なるほど」

「次はその Error Boundary を設置してるページコンポーネントを確認しよう」

リスト 48: src/routes/Members.tsx

```

import { Suspense } from "react";
import { ErrorBoundary } from "react-error-boundary";
import { Link, Navigate, useParams } from "react-router";
import { Loader2 } from "lucide-react";
import { toast } from "sonner";
import MemberList from "@/components/MemberList.tsx";
import OrgHeader from "@/components/OrgHeader.tsx";
import { Toaster } from "@/components/ui/sonner.tsx";
import MembersError from "../_MembersError.tsx";

export default function Members() {
  const { orgId } = useParams();

  if (!orgId) {
    return <Navigate to="/" replace />;
  }

  return (
    <>
      <ErrorBoundary
        FallbackComponent={MembersError}
        onReset={() => {

```

```

        toast("⚠️ 再試行されました");
    }}
  >
  <OrgHeader orgId={orgId} />
  <Suspense fallback={<Loading />}>
    <MemberList orgId={orgId} />
  </Suspense>
</ErrorBoundary>
<hr className="mb-4 mt-8" />
<div className="flex justify-center">
  <Link to="/">↑top トップへ戻る</Link>
</div>
<Toaster />
</>
);
}
:

```

「さっきの `<OrgHeader>` コンポーネントに加えて、実際に GitHub API を叩いてる `<MemberList>` コンポーネントを Error Boundary（エラー境界）の中に設置した。react-error-boundary が提供する `<ErrorBoundary>` コンポーネントの props 仕様は以下の通り」

- `onError` エラー捕捉時に実行されるコールバック関数
- `onReset` エラーをリセットしたときに実行されるコールバック関数
- `fallbackRender` エラー捕捉後の UI をレンダリングするための関数
- `FallbackComponent` エラー捕捉後に表示されるコンポーネント

「`fallbackRender` と `FallbackComponent` はまったく同じ用途の props なので好みに合わせて使えばいいけど、今回は別ファイルに切り出したコンポーネントを使いたいので `FallbackComponent` を採用した」

「`onReset` に渡してるコールバック関数の中でコールされてる `toast` というのは？」

「`FallbackComponent` にはエラーをリセットするための関数が引数として渡される。それを実行するとエラーがクリアして再レンダリングされるんだけど、実行されたことを知らせるために shadcn/ui の Sonner¹²⁵ を使ってる。画面右下にふわっと表示されて一定時間で消える UI ね。コンポーネントツリーの任意の場所に `<Toaster />` を設置した上で sonner パッケージの `toast` 関数を実行すると、トース

¹²⁵ <https://ui.shadcn.com/docs/components/sonner>

ト UI が表示されるわけ」

「ああ、そういうの最近よく見かけますね」

「じゃあ最後に `FallbackComponent` の値に設定されてる `MembersError` の実装を見ておこう」

リスト 49: `src/routes/_MembersError.tsx`

```
import type { FallbackProps } from "react-error-boundary";
import { Button } from "@components/ui/button.tsx";

export default function MembersError({
  error,
  resetErrorBoundary,
}: FallbackProps) {
  return (
    <div className="flex items-center justify-center py-20">
      {error instanceof Error && error.cause === 404 ? (
        <div className="flex items-center gap-6">
          <div className="text-2xl font-medium leading-none">404</div>
          <div className="h-16 w-px bg-border" />
          <div className="flex flex-col">
            <div className="text-xl">その組織は見つかりませんでした</div>
            <div className="text-lg text-gray-400">
              {error.message}
            </div>
          </div>
        </div>
      ) : (
        <div className="flex flex-col items-center gap-6">
          <div className="text-xl">
            API エラーが発生しました。時間を置いてから再試行してみてください。
          </div>
          <div className="flex space-x-4">
            <Button
              variant="outline"
              onClick={resetErrorBoundary}
              className="mt-4"
            >
              再試行する
            </Button>
          </div>
        </div>
      )}
    </div>
  );
}
```

「FallbackComponent コンポーネントは捕捉したエラーとリセット関数をそれぞれ `error` および `resetErrorBoundary` という引数として受け取ることができる。ただし提供されてる `FallbackProps` インターフェースではなぜか `error` が `any` 型で定義されてるので、プロパティにアクセスするには `Error` オブジェクトであることを確認する必要がある」

「404 以外のエラーのときに再試行ボタンが表示されて、そこに引数で受け取った `resetErrorBoundary` がトリガーされるようになってますね」

「API エラーが起きたときに再試行してもすぐ状況が改善するケースはあまりないだろうけど、今回はデモンストレーションのサンプルだからね。挙動を確認するためにこうしてみた。じゃあ開発サーバを起動してブラウザから `/foobar/members` や `/error/members` にアクセスしてみて」

「404 は問題なく表示されますね。 `/error/members` のほうはと……」



図 15: 意図的なエラーを発生させて表示したエラー画面

「あ、『再試行する』ボタンをクリックしたら、それを知らせるトーストが表示されました！」

「以上が `react-error-boundary` を使って SPA で Error Boundary を構築する方法ね。簡単でしょ？」

13-3. Next.js の App Router でエラーハンドリング

「次は Next.js で App Router を使ったときのエラーハンドリングのやり方ね。App Router によるアプリケーションでは Server Components と Client Components が混在するため、自前で Error Boundary を構築せずにもっぱらフレームワークで用意された規約ファイルを使う方法が推奨される」

「ん？ どういうことですか？」

「Server Components は関数コンポーネントしかサポートしないので、`componentDidCatch` や

`getDerivedStateFromError` は使えない。それを内部で利用している `react-error-boundary` も同様。だから Server Components で生じたエラーをそのコンポーネントの外で捕捉する手段が該当する規約ファイルを使う以外にないの。App Router でアプリケーションを構築する場合、ツリー内の大部分が Server Components で構成されることになるはずなので、素直にフレームワークの流儀に従ったほうがいいわけ」

「なるほど。規約ファイルによるしくみとしては、`loading.tsx` を置くと自動的に `Suspense` ブロックが構築されるというものがありましたよね。エラーについても該当するファイルを置くと `Error Boundary` が自動的に適用されると考えていいのでしょうか？」

「そのとおり。App Router の規約ファイルによるコンポーネントの階層構造の図をもう一度参照してみよう」

<code><Layout></code>	→ <code>layout.tsx</code>
<code><Template></code>	→ <code>template.tsx</code>
<code><ErrorBoundary fallback={<Error />}></code>	→ <code>error.tsx</code>
<code><Suspense fallback={<Loading />}></code>	→ <code>loading.tsx</code>
<code><ErrorBoundary fallback={<NotFound />}></code>	→ <code>not-found.tsx</code>
<code><Page /></code>	→ <code>page.tsx</code>
<code></ErrorBoundary></code>	
<code></Suspense></code>	
<code></ErrorBoundary></code>	
<code></Template></code>	
<code></Layout></code>	

「これ最初見たときあんまりわかりませんでしたけど、今ならかなり理解できます。`error.tsx` も `not-found.tsx` も `Error Boundary` の `fallback` コンポーネントに適用される。そして `Not Found` エラーはデータ取得処理に伴って起きるものだから `Suspense` ブロックの中に配置されてるんですね」

「そうだね。じゃあ実際のコードを見ていこうか¹²⁶。さっきと同様に `GitHub API` から取得したデータを `Suspense` で表示するバリエーションを改変していく。404 に関してははもとも対応済みなので、`/error/members` で意図的にエラーを発生させるルーチンだけを追加しておく」

リスト 50: `src/app/[orgId]/members/page.tsx` の差分

```

:
function generateTitle(orgId: string) {
+   if (orgId === "error") {
+     throw new Error('Intentional Error!');
+   }
  const org = getOrganization(orgId) || notFound();

```

¹²⁶ <https://github.com/klemiary/Riakuto-StartingReact-ja5.0/tree/main/13-error/03-app-router>

第 14 章 メモ化でレンダリングを効率化する

14-1. 計算結果をメモ化する

「実際の開発にあたってはアプリケーションの規模が大きくなるにしたがって、パフォーマンスチューニングが重要になってくる。React でパフォーマンスを悪化させるもっとも多い原因は、コンポーネントツリーに不要な再レンダリングが発生してしまうこと。ここまでのサンプルではあまりインタラクティブな機能がなかったけど、ユーザーからのアクションに素早く対応して UI を変化させるリッチなアプリケーションを作ろうとすると、再レンダリングの問題が顕著にパフォーマンスに影響してくるのね。だから React には不用意にコンポーネントの再レンダリングを起こさないようにしたり、再レンダリングされてもパフォーマンスの悪化を最小限にするための API がいくつも用意されている」

「ふむふむ」

「そのうちもっともよく用いられるのが**メモ化**（Memoization）を行う API。メモ化とはコンピュータサイエンスでは一般的な最適化のための技法で、任意のサブルーチンを呼び出した結果を後で再利用するために保持しておき、それが呼び出されるたびに再計算されることを防ぐというもの。コンポーネントのレンダリング結果だったり、コンポーネント内の高コストな計算結果だったりを保持しておくことでパフォーマンスの最適化を図るというのが React におけるメモ化なわけね。

「というわけで以前のおさらいとして、React のコンポーネントが再レンダリングされるための条件は何だったかを挙げてみてくれる？」

「えーっと、こんな感じじゃなかったでしょうか？」

- 自身の `props` が更新されたとき
- 自身の `state` が更新されたとき
- 親コンポーネントが再レンダリングされたとき

「はい正解。なお実はこれに加えて『参照している Context¹³¹ の値が変更されたとき』というのもあるんだけど、React を SPA としてではなくメタフレームワーク経由で使うことがほとんどの現在、Context の出番はほとんどないのでこれについては省略する。挙げてくれた 3 つの中でそれを減らすともっともパフォーマンスの向上に貢献するのは、何とんでも親コンポーネントが再レンダリング

¹³¹ 「コンテキストで深くデータを受け渡す - React」
<https://ja.react.dev/learn/passing-data-deeply-with-context>

されたときに連動して自身も再レンダリングするというものだね。これによって『ぶよぶよ』のように再レンダリングの連鎖反応が次々に起きてしまう」

「ぶよぶよ……、なつかしいですね。でもイメージとしてはすぐピンときます」

「だから子コンポーネントをメモ化しておき、親コンポーネントが再レンダリングされても連鎖せずにそのメモ化されたコンポーネントを返すための API が用意されてる。名前もまんま `memo()` として¹³²、使い方は以下の通り」

```
memo(
  Component: FunctionComponent<P>,
  propsAreEqual?: (prevProps: P, nextProps: P) => boolean,
);
```

「第 1 引数にコンポーネント、オプションの第 2 引数で `props` のカスタム比較関数を取り、結果としてメモ化されたコンポーネントを出力する関数ね」

「こんな API があったんですね。でもこれを活用しようとするとのコンポーネントに使うべきかの判断も難しそうだし、片っぱしから使っていくとコードがかなり読みづらくなったりしませんか？」

「その懸念はごもっとも。だからこの `memo()` は 2018 年 10 月リリースのバージョン 16.6 のときから存在してるんだけど、あまり積極的には使われてこなかった。それが最近になってブレイクスルーがあって状況が一変したんだけど、その話はメモ化の API をひとつと説明した後でしょう」

「わかりました」

「次に紹介するのは `useMemo()`¹³³。これはレンダリングを抑制するのではなく、コンポーネント内で時間のかかりそうなロジックの処理結果をメモ化することで再レンダリングのコストを下げるためのもの。具体的なコードを使って説明しようか。Custom Hooks で作ったカウントダウンタイマーに、残りカウントが素数のときだけその文字色をピンクにする機能をつけ加えてみる¹³⁴。まずは素数のリストを取得するユーティリティ関数を作成しておく」

リスト 56: `src/lib/math.ts`

```
export function getPrimeNumbers(max: number) {
  if (max < 2) return [];

  return [...Array(max + 1).keys()].slice(2).filter((n) => {
    for (let i = 2; i < n; i += 1) {
```

¹³² 「memo - React」 <https://ja.react.dev/reference/react/memo>

¹³³ 「useMemo - React」 <https://ja.react.dev/reference/react/useMemo>

¹³⁴ <https://github.com/kleмиwary/Riakuto-StartingReact-ja5.0/tree/main/14-memoize/01-memo>

```

    if (n % i === 0) return false;
  }

  return true;
});
}

```

「なるほど。REPL で `getPrimeNumbers(10)` を実行してみたら、`[ 2, 3, 5, 7 ]` がちゃんと返ってきますね」

「これを普通に `Timer` コンポーネントに導入して想定する動作になるように書き換えると以下のようになるはず」

```

import { useMemo } from "react";
import { RotateCw } from "lucide-react";
import { Button } from "@components/ui/button.tsx";
import { Card, CardContent, CardHeader } from "@components/ui/card.tsx";
import { useTimer } from "@hooks/use-timer.ts";
import { getPrimeNumbers } from "@lib/math.ts";

interface TimerProps {
  maxCount?: number;
}

export default function Timer({ maxCount = 60 }: TimerProps) {
  const [countLeft, reset] = useTimer(maxCount);
  const primeNumbers = getPrimeNumbers(maxCount);

  return (
    <Card className="w-80 shadow-md">
      <CardHeader className="pb-2 pt-4">
        <div className="text-center text-xl font-medium">Count</div>
      </CardHeader>
      <CardContent className="flex justify-center py-1">
        <div className="text-4xl font-semibold">
          {primeNumbers.includes(countLeft) ? (
            <span className="text-pink-400">{countLeft}</span>
          ) : (
            <>{countLeft}</>
          )}
        </div>
      </CardContent>
      <CardContent className="mx-4 flex pb-6 pt-4">
        <Button className="w-full bg-red-500 hover:bg-red-600" onClick={reset}>
          <RotateCw className="mr-2 size-4" /> Reset
        </Button>
      </CardContent>
    </Card>
  );
}

```

```

    </Button>
  </CardContent>
</Card>
);
}

```

「この例では再レンダリングのたびに毎回 `getPrimeNumbers()` が呼び出されて素数リストを出力し、それを `primeNumbers` に代入してる。実際の `getPrimeNumbers()` はそこまで重いルーチンではないけど、再帰的な処理や全探索を行うようなものでは前提条件によって指数関数的に計算コストが増大し、体感レベルでパフォーマンスを悪化させてしまうことがある。だから `useMemo()` を使って計算結果をメモ化してあげるの」

リスト 57: `src/components/Timer.tsx` の差分

```

export default function Timer({ maxCount = 60 }: TimerProps) {
  const [countLeft, reset] = useTimer(maxCount);
-  const primeNumbers = getPrimeNumbers(maxCount);
+  const primeNumbers = useMemo(() => getPrimeNumbers(maxCount), [maxCount]);
  return (

```

「なんか使い方が `useEffect` と似てますね」

「第1引数がメモ化させたい計算結果を出力する関数、第2引数が依存配列ね。ここにその値が変更されていれば再計算を走らせたい変数を格納する。空配列を渡すと初回のレンダリング時にのみ計算される。`useEffect` とちがうのは依存配列を省略できないこと。再レンダリングのたびに計算されるならメモ化する意味がないからね」

「なるほど。で、この場合はコンポーネントの props である `maxCount` が変わったときに再計算するようになってるわけですね」

「そう。では開発サーバを起動して動作を見てみよう」

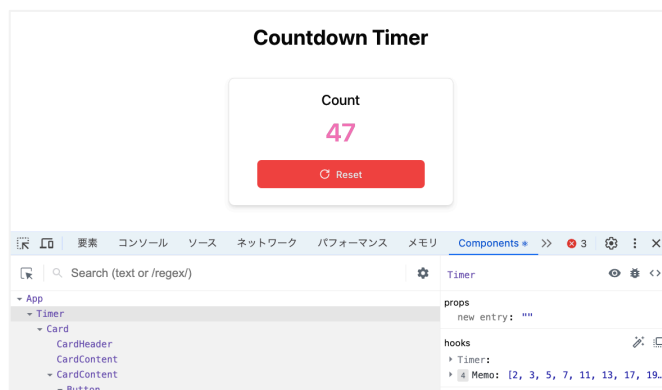


図 16: 素数でピンクになるカウントダウンタイマー

「あ、問題なく素数のときだけアホ.....じゃなくてピンクになりますね！」

「メモ化された値は React Developer Tools から確認できるので、開発時には必要に応じて参照するといいよ」

14-2. 関数定義をメモ化する

「ところでこのサンプル、タイマーとしては不完全なところがある。数字が切り替わってから、0.9 秒後くらいのタイミングでリセットボタンを押してみてくれる？」

「難しい注文ですね。.....今だ！ あれ？ 60 に戻ったとたん、1 秒も経たずぐ 59 になっちゃいました」

「うん。もともとの `useTimer()` の実装は以下の通りだったよね」

リスト 58: `src/hooks/use-timer.pre.ts`

```
import { useEffect, useState } from "react";

export function useTimer(maxCount: number) {
  const [countLeft, setCountLeft] = useState(maxCount);

  function tick() {
    setCountLeft((c) => c - 1);
  }

  function reset() {
    setCountLeft(maxCount);
  }

  useEffect(() => {
    const timerId = setInterval(tick, 1000);

    return () => clearInterval(timerId);
  }, []);

  useEffect(() => {
    if (countLeft === 0) {
      setCountLeft(maxCount);
    }
  }, [countLeft, maxCount]);
}
```



```
return [countLeft, reset] as const;
}
```

「reset() が実行された際、単純にカウント数を元の値に戻してるだけなのでさっきのような挙動になるわけね。この違和感を解消するには、リセットボタンが押されたときにインターバルを新しく作り直す必要がある。ただ reset 関数の中でインターバル実行をクリアしたくても、インターバル ID がひとつめの useEffect のセッアップ関数のスコープに閉じ込められていて参照できない」

「ほんとだ、どうすればいいんでしょう？ あ、state にしたらどうですか？」

「ううん、コンポーネントのレンダリングに直接関係しない値は基本的に state にするべきじゃない。依存配列の関係を不用意に複雑にできてしまっただけでバグの原因になりかねないからね。こういう場合は useRef¹³⁵ を使うといい」

「ゆーずれふ？」

「ref とはコンポーネントに対応する DOM に直接アクセスするための手段として React が提供しているもので¹³⁶、useRef() は本来その参照値を生成するために作られた Hooks API。でもレンダリングに影響を与えない一時的な値の保存に使うこともできるの。この useRef() を使って timerId を取り回すように use-timer.ts を変更しよう」

リスト 59: timerId を useRef() で保持するように変更

```
- import { useEffect, useState } from "react";
+ import { useEffect, useRef, useState } from "react";

export function useTimer(maxCount: number) {
  const [countLeft, setCountLeft] = useState(maxCount);
+  const timerId = useRef<NodeJS.Timeout>(null);

  function tick() {
    setCountLeft((c) => c - 1);
  }

  function reset() {
+    if (timerId.current) {
+      clearInterval(timerId.current);
+    }

+    timerId.current = setInterval(tick, 1000)
    setCountLeft(maxCount);
  }
}
```

¹³⁵ 「useRef - React」 <https://ja.react.dev/reference/react/useRef>

¹³⁶ 「Ref と DOM - React」 <https://ja.legacy.reactjs.org/docs/refs-and-the-dom.html>

第 15 章 フォームでデータを更新する

15-1. React でフォームの UI を実装するには

15-1-1. プリミティブな React フォーム

「ここまで学んだことで、閲覧がメインの基本的な Web アプリケーションであればもう作れるようになったはず。でもまだ商業レベルのアプリケーションを構築するのに大事な機能が欠けてるよね」

「はい。外部データの取得は教えてもらいましたが、その更新のやり方がまだでした。では次の課題はフォームの実装とそれによるデータ更新ですか？」

「そう。ただ React でフォームを扱うのはけっこう難儀だね。クライアントサイドでインタラクティブな UI を実現するためにはフォーム本来の動作を上書きして任意のロジックを実行させる必要がある。Ruby on Rails のように標準の HTML フォームとそれによるオーソドックスなデータ送信に慣れてるとだいぶ煩雑に感じんじゃないかな。さらに例のごとくフォーム周りのライブラリも百花繚乱で、デファクトスタンダードが定まらない」

「ふむふむ」

「使いこなせるようになるまでは遠い道のりだけど、基本からひとつひとつ学んでいこう。まず React でフォームを作成するには組み込み API の `<form>` コンポーネントを利用する。これまでカウンターやタイマーのサンプルアプリでは入力要素が単一だったので `<button>` コンポーネントの `onClick` にイベントハンドラを設定することでインタラクティブな UI を実現してた。でも複数の入力項目がありそれらの内容をボタン押下でまとめてハンドリングしたい場合は `<form>` コンポーネントが必要になる」

「はい。そこは標準的な HTML と同じですね」

「問題はここから。React アプリケーションでフォームデータを扱う際は最低限、次の 2 つのことを考えて適切な手法を選択する必要がある」

- ・ フォームの入力データを何によって保持・管理するか
- ・ データをどこに `submit` してどのようにハンドリングするか

「.....ちょっとどういうことかわからないんですが？」

「実際のコードを見ないとピンとこないかもね。たとえばこういう見た目のフォームを実現したいとする」



図 19: ユーザー情報登録フォーム

「これを旧来型の SPA で、かつライブラリを使わずに実装したサンプルを用意した¹⁴⁷。そのフォーム部分のコンポーネントは次のようなコードになる」

リスト 65: 03-form/raw-form/src/components/RegistrationForm.tsx

```
import { useState } from "react";
import { Button } from "@components/ui/button.tsx";
import { Card, CardContent } from "@components/ui/card.tsx";
import { Checkbox } from "@components/ui/checkbox.tsx";
import { Input } from "@components/ui/input.tsx";
import { Label } from "@components/ui/label.tsx";
import {
  Select, SelectContent, SelectItem, SelectTrigger, SelectValue,
} from "@components/ui/select.tsx";

const genderOption = { m: "男性", f: "女性", n: "それ以外" } as const;

interface RegData {
  username: string;
  zipcode?: string;
  gender?: keyof typeof genderOption;
  isAgreed: boolean;
}

export default function RegistrationForm() {
  const [regData, setRegData] = useState<RegData>({
```

¹⁴⁷ <https://github.com/kleimiary/Riakuto-StartingReact-ja5.0/tree/main/15-form/01-react-form/01-primitive>

```

    username: "",
    isAgreed: false,
  });

function handleSubmit(event: React.FormEvent<HTMLFormElement>) {
  event.preventDefault();
  console.log(regData);
}

function handleInput(event: React.ChangeEvent<HTMLInputElement>) {
  const { name } = event.target;
  const value = event.target.value;
  setRegData((state) => ({ ...state, [name]: value }));
}

return (
  <Card className="w-md max-w-md p-5 shadow-md">
    <CardContent className="px-1 py-2">
      <form onSubmit={handleSubmit}>
        <fieldset className="mb-4">
          <Label className="mb-2 block">ユーザー名 ( 必須 ) </Label>
          <Input
            name="username"
            value={regData.username}
            onChange={handleInput}
            className="w-full"
          />
        </fieldset>
        <fieldset className="mb-4">
          <Label className="mb-2 block">郵便番号</Label>
          <Input
            name="zipcode"
            maxLength={8}
            value={regData.zipcode ?? ""}
            onChange={handleInput}
            className="w-full"
          />
        </fieldset>
        <fieldset className="mb-6">
          <Label className="mb-2 block">性別</Label>
          <Select
            value={regData.gender}
            onChange={(value) => {
              const gender =
                value in genderOption
                  ? (value as keyof typeof genderOption)
            }}
          />
        </fieldset>
      </form>
    </CardContent>
  </Card>
);

```

```

        : undefined;
        setRegData((state) => ({ ...state, gender }));
    }}
>
<SelectTrigger>
  <SelectValue placeholder="選択してください" />
</SelectTrigger>
<SelectContent>
  {Object.entries(genderOption).map(([code, name]) => (
    <SelectItem value={code} key={code}>
      {name}
    </SelectItem>
  ))}
</SelectContent>
</Select>
</fieldset>
<Label
  htmlFor="isAgreed"
  className="mb-4 flex items-center justify-center"
>
  <Checkbox
    id="isAgreed"
    checked={regData.isAgreed}
    onCheckedChange={(checked) => {
      const isAgreed = checked === "indeterminate" ? false : checked;
      setRegData((state) => ({ ...state, isAgreed }));
    }}
    className="mr-2"
  />
  <span>規約に同意する</span>
</Label>
<fieldset className="flex justify-center pt-2">
  <Button
    type="submit"
    className="w-2/3 bg-blue-500 text-white hover:bg-blue-400"
    disabled={!regData.isAgreed && !!regData.username.trim()}
  >
    送信
  </Button>
</fieldset>
</form>
</CardContent>
</Card>
);
}

```

「な、長い……」

「フォームの実装は入力項目が多いからどうしても長くなるのよ。でもさっき提示したポイントに絞って読み解いていけばだいじょうぶ。まずフォームの入力データはどうやって管理されてる？」

「RegData というインターフェースで定義されたオブジェクトを state として用意して、そのプロパティを各入力要素で参照してるんじゃないですか？」

「そう。通常の HTML フォームと違って入力値の読み書きを React の制御下に置いてるの。ここではコンポーネントの state を用いて、たとえばユーザー名の入力要素では onChange プロパティでその更新をトリガーし、value でその値を参照してる。だから onChange の値に設定してるイベントハンドラを削除すると、テキストに何も入力できなくなる」

「ほんとだ。ふーむ、柴崎さんが言いたかったことがなんとなくわかってきました。正しく理解したのでひとつずつ質問させてくださいね。まず onChange というプロパティは初出だと思うので、くわしく教えてください」

「shadcn/ui の <Input> コンポーネントは React 組み込みの <input> を拡張したもので、props もほぼ共通してる。onChange は入力値が変更されたときに呼び出されるイベントハンドラを設定するためのものね。ここでは関数 handleInput を渡してるけど、その中で setRegData() を使って username を更新してるわけね」

「handleInput() の引数に渡されてるのは何ですか？ 型定義もすごい摩訶不思議ですけど」

「onChange にはコールバック関数を渡すようになってるんだけど、発生した change イベントがその引数になるの。JavaScript では DOM 要素で発生したイベントは Event オブジェクト¹⁴⁸によって表現される。ただし React ではネイティブの Event ではなく、独自の SyntheticEvent オブジェクト¹⁴⁹が用いられる。その理由はブラウザごとの差異を吸収したり、仮想 DOM との連携で最適化をするためね。ChangeEvent<T> は change イベントの SyntheticEvent バージョンで、型引数<T>に HTMLInputElement を渡すことで <input> コンポーネントの change イベントの型が表現される。onChange に設定するイベントハンドラ関数には引数としてこのイベントが渡されるわけ」

「めっちゃ複雑ですね……。どのイベントにどの型を適用するか、どうやったらわかるんですか？」

「公式ドキュメントにはもちろんないので、@types/react による型定義のコード¹⁵⁰を直接を見るしかない。他には React TypeScript Cheatsheet というドキュメントにもイベント型の一覧がある¹⁵¹。あと

¹⁴⁸ 「Event - Web API | MDN」 <https://developer.mozilla.org/ja/docs/Web/API/Event>

¹⁴⁹ 「合成イベント (SyntheticEvent) - React」
<https://ja.legacy.reactjs.org/docs/events.html>

¹⁵⁰ 「DefinitelyTyped/types/react/index.d.ts • DefinitelyTyped/DefinitelyTyped」
<https://github.com/DefinitelyTyped/DefinitelyTyped/blob/master/types/react/index.d.ts>

¹⁵¹ 「Forms and Events | React TypeScript Cheatsheets」
https://react-typescript-cheatsheet.netlify.app/docs/basic/getting-started/forms_and_events/

裏ワザとして、VS Code 上で適当なアロー関数をインラインで書いてその引数にマウスホバーすると推論された型が表示されるよ」

```
<Input
  id="username"
  value={for (parameter) event: React.ChangeEvent<HTMLInputElement>
  onChange={event => console.log(event)}
  className="mb-4 w-full"
/>
```

図 20: インライン関数の引数にマウスホバー

「あ、これいいですね。でもそれなら、最初からインラインでイベントハンドラを書けばよかったんじゃない？」

「handleInput は zipcode の onChange にも渡してるでしょ。イベントから id を抽出して、それを state 更新のキーにすることで汎用的に作ってるの。また複数行に渡る長い関数をインラインで書くと可読性が悪くなるので、インラインにするかどうかはバランスを考えて決めてね」

「<Select> コンポーネントと <Checkbox> コンポーネントではインラインで書いてますよね」

「<Select> の onChange と <Checkbox> の onChange は shadcn/ui のコンポーネントで実装されているイベントハンドラ設定用の props だね。これらのコールバックには引数に直接、入力値の value が渡されるので型の整合を取ったうえで state に反映させてる。

入力データを state により唯一の情報源として保持し、更新をそのセット関数を通じてのみ行うフォームのコンポーネントを **Controlled Components**（制御コンポーネント）と呼ぶこともある¹⁵²。

React でフォームを構築する際の一般的な手法だね」

「ということは制御されてないコンポーネントもあるということですか？」

「そう。入力データを state で管理すると頻繁にコンポーネントの再レンダリングが起きてしまってパフォーマンスが悪いため、最近のフォームライブラリでは内部で state を直接には用いないことが多い。それらでは値の参照には ref プロパティを用いて <input> などの DOM に直接アクセスしたり、ネイティブの FormData¹⁵³ オブジェクトを使ったりしている。値の保持はシンプルに DOM に任せたり、Proxy オブジェクト¹⁵⁴で変更を追跡しつつその保持自体は useRef() で行ったり」

「そうか、state で管理すると変更があるたびにコンポーネントが再レンダリングされちゃうんです

¹⁵² 「フォーム - React」 > 「制御されたコンポーネント」

<https://ja.reactjs.org/docs/forms.html#controlled-components>

¹⁵³ 「FormData - Web API | MDN」

<https://developer.mozilla.org/ja/docs/Web/API/FormData>

¹⁵⁴ 「Proxy - JavaScript | MDN」

https://developer.mozilla.org/ja/docs/Web/JavaScript/Reference/Global_Objects/Proxy

ね。ならこのサンプルアプリでは、テキストフォームに 1 文字入力するごとにコンポーネント全体が再レンダリングされてるわけですか？」

「そう。React Developer Tools で再レンダリングのコンポーネントをハイライトするようにすると、その挙動が確認できるよ。ただ自前で安全な非制御コンポーネントのしくみを用意するのは工数がかかるので、ライブラリを使わない場合は制御されたコンポーネントにするのが一般的だね」

「場面によって取るべき戦術がちがってくるというのはそういうことですか。React でフォームの扱いが難儀というのがわかってきました……」

「React でのフォームにおける 2 つめのテーマは、データをどこに submit してどのようにハンドリングするかということだったね。ここでは `<form>` コンポーネントの `onSubmit` プロパティでフォームが送信された submit イベントをハンドリングしてる。React でアプリケーションを SPA として構築する際にはこれがメジャーなやり方かな」

「イベントハンドラとして設定されている `handleSubmit` 関数で state の `formData` をコンソールに出力してますね。その直前の行は何をやってるんですか？」

「Event オブジェクトの `preventDefault()` メソッドは、そのイベント本来の振る舞いを阻止するもの¹⁵⁵。通常はフォームを submit すると、`action` 属性で指定されている URL、それがなければ現在の URL にフォームの値が POST で送られてページが遷移しちゃうでしょう。ここでそれをやるとアプリケーションが再読み込みされて、state もすべてクリアされてしまう。それをさせないためにこの 1 行を入れているわけ」

「なるほど。この行をコメントアウトしたら、たしかにその挙動になりますね」

「ちなみにこれまでのサンプルで `<button>` コンポーネントの `onClick` プロパティに渡してたコールバックも、実は似たような措置が必要なの。安全性を考えれば、本当はこんなふうに書かないといけなかった」

```
function reset(event: SyntheticEvent) {
  event.stopPropagation();
  setCountLeft(maxCount);
}
```

「`stopPropagation()` って何ですか？」

「click を始めとする DOM イベントはデフォルトで親要素、さらにその親要素へと階層的に伝播するという性質を持ってる。たとえば親要素と子要素にそれぞれ click のイベントリスナーを設定してあると、子要素のクリックで親要素のイベントハンドラが連動して実行されてしまう。イベント発火が下から泡のように浮き上がってくるのでこの挙動は**イベントバブリング** (Event Bubbling) と呼ばれ

¹⁵⁵ 「Event: `preventDefault()` メソッド - Web API | MDN」
<https://developer.mozilla.org/ja/docs/Web/API/Event/preventDefault>

る¹⁵⁶。

これを防ぐために、`Event` オブジェクトには `stopPropagation()` メソッドが用意されてるの。
`propagation` とは電波や音波の伝播とか、思想や習慣が広まること。UI で `click` イベントを利用する際は、誤動作を防ぐためにこのメソッドを使うことが推奨される」

「イベントにはそんな挙動があったんですね。知りませんでした」

「`handleSubmit()` のメインロジックに話を戻そう。このプロジェクトは簡易サンプルなので、入力されたデータをただコンソールにダンプしてるだけで済ませてる。でも実際のアプリケーションでは値のバリデーションを行ったうえで外部 API にデータを送信し、そのレスポンスを受け取ったうえで正常系および異常系の UI 遷移を用意しないといけない」

「入力値のバリデーションをクライアント側でもやる必要があるんですか？ サーバ API でも当然やるはずだから二度手間じゃないですか？」

「通信にはラグがあるし、API への無駄なリクエストも減らしたいので React で作るようなリッチなアプリケーションではクライアントでもバリデーションをするのが一般的だね。ただこれも自前で実装するのは複雑すぎるので、実際には何かしらフォームライブラリを使う必要に迫られるんだけど」

「じゃあ現場ではライブラリなしにフォームを使うケースはほとんどないってことですか」

「まあそうだね。ただ今は基本を学んでいるところなので、ライブラリを使った書き方はその後であらためて説明する。また React のフォームでは `onSubmit` に設定したイベントハンドラで送信データを扱う以外の手法があるんだけど、その紹介も少し待ってね。先にフォームのステータスの管理について学んでおこう」

15-1-2. フォームのステータスを管理する

「フォームのステータスって何ですか？」

「フォームを送信すると通信が発生するため多少のラグが発生する。するとレスポンスがないと勘ちがいしてボタンを連打して重複送信したりするユーザーが少なからずいる。だから送信中は重ねてボタンを押せなくしたり、送信中であることを示す UI を表示したりする必要があるの」

「なるほど」

「ゆえに今回は次のようなフォームを構築する。送信中はボタンが無効化されて押せなくなり、テキストが『送信中……』になる。またコードをシンプルにするために入力項目は 2 つに減らした」

¹⁵⁶ 「イベントのバブリング - ウェブ開発の学習 | MDN」

https://developer.mozilla.org/ja/docs/Learn_web_development/Core/Scripting/Event_bubbling

図 21: ユーザー情報登録フォーム

「ではサンプルのアプリケーション¹⁵⁷を起動してみてくれる？ トップページに『state で管理しているフォーム』『useTransition で管理しているフォーム』『子コンポーネントで useFormStatus を使ったフォーム』という3つのリンクが表示されてるよね。見た目はほぼ同じだけど実装が少しずつ異なる。ひとつめの『state で管理しているフォーム』のコードからチェックしていこう」

リスト 66: src/routes/StateForm.tsx

```
import { useState } from "react";
:
export default function StateForm() {
  const [regData, setRegData] = useState<RegData>({ username: "" });
  const [isPending, setIsPending] = useState(false);

  async function handleSubmit(event: React.FormEvent<HTMLFormElement>) {
    event.preventDefault();
    setIsPending(true);

    try {
      await registerUser(regData);
      toast("🎉 登録されました");
    } catch (_err) {
      toast("⚠️ 不正な入力エラー");
    } finally {
      setIsPending(false);
    }
  }
}
```

¹⁵⁷ <https://github.com/klemiary/Riakuto-StartingReact-ja5.0/tree/main/15-form/01-react-form/02-form-status>

```

function handleInput(event: React.ChangeEvent<HTMLInputElement>) {
  const { name } = event.target;
  const value = event.target.value;
  setRegData((state) => ({ ...state, [name]: value }));
}

return (
  <Card className="w-md max-w-md p-5 shadow-md">
    <CardContent className="px-1 py-2">
      <form onSubmit={({data}) => void handleSubmit(data)}>
        <fieldset className="mb-4">
          <Label className="mb-2 block">ユーザー名 ( 必須 ) </Label>
          <Input
            name="username"
            value={regData.username}
            onChange={handleInput}
            className="w-full"
          />
        </fieldset>
        <fieldset className="mb-4">
          <Label className="mb-2 block">郵便番号</Label>
          <Input
            name="zipcode"
            maxLength={8}
            value={regData.zipcode ?? ""}
            onChange={handleInput}
            className="w-full"
          />
        </fieldset>
        <fieldset className="flex justify-center pt-4">
          <Button
            type="submit"
            className="w-2/3 bg-blue-500 text-white hover:bg-blue-400"
            disabled={isPending || !regData.username.trim()}
          >
            {isPending ? "送信中..." : "送信"}
          </Button>
        </fieldset>
      </form>
    </CardContent>
  </Card>
);
}

```

「isPending という state 変数を用いて送信状態を管理してるわけですね。それによってさっき説明してもらった UI が実現されてる」

「そう。handleSubmit() で最初に isPending を true にして、API をコールする try ブロックの finally でそれを false に戻してる」

「registeruser() では何をしてるんですか？」

「実装を見ればすぐわかるよ。1 秒スリープしてから郵便番号が数値として解釈不能なら throw し、そうでなければフォームデータをコンソールにダンプする。なお registeruser が非同期関数なのでそれをコールする handleSubmit も非同期関数にする必要があった。だから onSubmit にイベントハンドラとして設定するのに戻り値の Promise を無視することを明示するために (data) => void handleSubmit(data) という記述にしてる。こうしないとこの環境では ESLint エラーになるからね
158」

「ふむふむ」

「一般的にフォームで取り扱うステータスは、送信状態とエラーくらい。ただ現実のアプリケーションでは、エラーについてはどこでどんな種類のエラーが起こったのかによって表示する UI を変える必要があったりするのでこんなに単純ではないけどね。

じゃあ次はこの送信状態を別の方法で管理してみよう。リンクでは『useTransition で管理しているフォーム』、コードは以下の通り」

リスト 67: src/routes/TransitionForm.tsx

```
import { useState, useTransition } from "react";
:
export default function TransitionForm() {
  const [regData, setRegData] = useState<RegData>({ username: "" });
  const [isPending, startTransition] = useTransition();

  function handleSubmit(event: React.FormEvent<HTMLFormElement>) {
    event.preventDefault();
    startTransition(async () => {
      try {
        await registerUser(regData);
        toast("🎉 登録されました");
      } catch (_err) {
        toast("⚠️ 不正な入力エラー");
      }
    });
  }
}
```

158 「no-misused-promises | typescript-eslint」
<https://typescript-eslint.io/rules/no-misused-promises/>

```
function handleInput(event: React.ChangeEvent<HTMLInputElement>) {
  const { name } = event.target;
  const value = event.target.value;
  setRegData((state) => ({ ...state, [name]: value }));
}

return (
  <Card className="w-md max-w-md p-5 shadow-md">
    <CardContent className="px-1 py-2">
      <form onSubmit={handleSubmit}>
        <fieldset className="mb-4">
          <Label className="mb-2 block">ユーザー名 (必須) </Label>
          :
        </fieldset>
      </form>
    </CardContent>
  </Card>
)
```

「isPending を state ではなく `useTransition()` という関数の戻り値にしていますね。この関数は何ですか？」

「まず **Transition** の概念から説明するね。これは英語で『遷移』や『移行』という意味だけど、React の文脈では優先度の低い状態更新プロセスのことを指す。React 18 で Concurrent Rendering¹⁵⁹ が導入され、コンポーネントツリーのレンダリングはその優先度に基づいて処理がスケジューリングされ非同期で行われるようになった。そこでは UI 更新は次の 2 つに区別される¹⁶⁰」

- **Urgent Updates** 後続処理をブロックしてでも即座に反映される、緊急度の高い更新
- **Transition Updates** 後続処理をブロックせず場合によってはその反映が遅延される、緊急度の低い更新

「従来は Urgent Updates のみで、デフォルトではこちらが適用される。18 で Transition Updates が追加されたわけだけど、`useTransition()` はこれを適用するための Hooks API なのね。戻り値はタプルになっていて、そのひとつめは Transition が保留中であるかどうかを示すフラグ。2 つめは引数で渡されたコールバック内での更新を Transition Updates にするための関数。外部 API へのデータの送信はテキスト入力やボタンクリックとちがって、ユーザーが多少のタイムラグを覚悟する操作なので Transition Updates を適用するのに打って付けと言える。`useTransition()` は送信状態の管理のための API というわけではないけれども、公式ドキュメントでもサンプルコードの多くはその用途に使われ

¹⁵⁹ 「12-1-1; React における外部データ取得手法の変遷」を参照のこと。

¹⁶⁰ 「React v18.0 – React」 > 「新機能：トランジション」
<https://ja.react.dev/blog/2022/03/29/react-v18#new-feature-transitions>

てる¹⁶¹」

「送信状態の管理というのはひとつの用途ではあるけど、本来はユーザーの操作や計算結果などを画面に反映させる優先度を下げるためのものなんですね。他にはどんな使い道があるんですか？」

「インクリメンタルサーチやフィルタリング、タブの切り替えといったところかな。シーンによって結果が逐次反映されるのに違和感を抱いたり、それがパフォーマンスを悪化させるようなことがあればそこで使うことを検討すればいいよ」

「なるほど」

「なお `useTransition()` は React 18 で導入されたのだけれど、非同期関数がサポートされたのは 19 から。だから既存のコードでは送信状態の管理に使われている例は少ない。でも新しく書く場合は `useTransition()` で管理したほうがいいと思う。不必要に UI をブロックしない、余計な state を増やさずに済むといったメリットがあるから」

「わかりました」

「最後、トップページに戻って『子コンポーネントで `useFormStatus` を使ったフォーム』をクリックしてくれる？」

「あ、なんかフォームの中に猫がいますね。送信したらぐるぐる回りました」

図 22: 送信すると猫が回るフォーム

「この猫の絵文字のパーツは `<StatusCat>` というコンポーネントをマウントしてる。その部分の実装を見てみよう」

リスト 68: `src/components/StatusCat.tsx`

¹⁶¹ 「`useTransition` - React」
<https://ja.react.dev/reference/react/useTransition>

```
import { useFormStatus } from "react-dom";

export default function StatusCat() {
  const { pending } = useFormStatus();

  return (
    <div className="mb-6 flex justify-center text-5xl">
      {pending ? (
        <div className="h-10 w-10 animate-spin">🐱</div>
      ) : (
        <div className="h-10 w-10">🐱</div>
      )}
    </div>
  );
}
```

「あれ？ コンポーネントに props が渡されてませんね。てっきり `isPending` が渡されてるのだと思ってました」

「そう。 `useFormStatus()` はそのコンポーネントが属するフォームについての情報を取得してくれる Hooks API なのね。送信状態はその戻り値のプロパティ `pending` によって示される。なおこの API は DOM 固有の機能に依存しているため `react-dom` パッケージで提供されているので、インポートするときは気をつけてね」

「そうなんですね。ちなみに `pending` 以外はどんな情報が取得できるんですか？」

「`useFormStatus()` の戻り値は以下のようなインターフェースで表現される」

```
{
  pending: boolean;
  data: FormData | null;
  method: string | null;
  action: string | ((formData: FormData) => void | Promise<void>) | null;
}
```

「へーえ。ステータスだけでなく送信データも含めて、フォームにまつわるあらゆる情報を参照できるんですね」

「まあ実際には `pending` 以外の情報を活用する場面はあまりなさそうだけどね。コンポーネントの独立性を高めるので、`useFormStatus()` が使える場面では使っていこう。当たり前だけど `<form>` ブロック内に置いたコンポーネントでしか有効に働かないので、該当コンポーネントを置く場所には注意してね」

あとがき

『りあクト!』シリーズも本書でなんと改訂第5版となりました。初版から6年以上たってもまだ書き続けているとは、当時の筆者にはまったく考えもしないことです。反響の大きさにも驚いており、カンファレンスやミートアップで面識のない方々から「著書読みました。すごく助かってます!」というお声をいただくことが多く恐縮です。とあるセッションでは聴衆の中から『りあクト!』を読んだことがある方に挙手をお願いしたところ、実に3分の2くらい手が挙がって感動しました。

しかしいっぽうで、既存の内容が時代に合わなくなっていることを感じていました。React 19 に導入される新機能、Next.js の急速なシェア拡大などを鑑みると、もはや SPA 構築を前提とした本では読者に価値を感じてもらえなくなっているのではという懸念です。そこで今回の改訂にあたり、第2部と第3部の内容をほぼ書き直すことに決めました。第3部にいたっては、前版とは章の構成もまったく異なる別の本になっています。

そのため改版作業には、およそ1年半というこれまででもっとも長い期間を要しました。そして執筆中には React 19 の新機能の仕様が変更されたり、Remix が React Router になったり、ESLint が Flat Config をデフォルトにしたり、Tailwind CSS が破壊的アップデートを行うなど数々の予期せぬ事件が起きました。そのたびに前の章に戻っては書き終えたはずの内容やサンプルコードを修正するという、賽の河原の石積みのような作業をしいられたのは今となってはいい思い出(?)です。生みの苦しみは大きかったですが、その甲斐あってまさに今の React 開発の現場で活用してもらえる本に仕上がったと満足しています。

本書の校正には Remix Tokyo および React Tokyo のコミュニティでお声がけして、以下の方々にご協力いただきました。Coji Mizoguchi さん、Kosuke Ito さん、nkzwsh00 さん、takatocordelia さん、cordelia さん。厚く感謝いたします。

読者の方々におかれましては、読み終わった後にでもぜひ感想などをお聞かせいただけるとありがたいです。ブログや Twitter、ショップでのレビューなどをいただければ、必ず目を通します。X (旧 Twitter) では #りあクト というハッシュタグを用意していますので、こちらをお使いいただければより確実に筆者に届きます。

19 リリースにより数年ぶりのメジャーアップデートがあったところなので、React 本体にはしばらく大きな変化はないでしょう。しかし RSC と Server Functions がかつての Hooks のようにスムーズに受け入れられていくのか、Next.js 一強状態のメタフレームワークで React Router がどれだけ迫れるのかなどは注視していきたいところです。さらに AI 支援コーディングの普及により、私たちの仕事のやり方も大きく変化していくでしょう。本書の内容が、できるだけ長くお役に立てることを望みます。

著者紹介

大岡由佳（おおおか・ゆか）

国内の IT 企業数社での PHP や Ruby による開発およびプロダクトマネージャを経て、2016 年からフリーランスに転身。当時まだ日本ではマイナーな存在だった React に惚れ込み、React 専門のフリーランスエンジニアとして複数の現場を渡り歩く。その経験を元に執筆した「りあクト！」シリーズが評判を呼びヒット作となる。現在は事実上おそらく国内（世界でも？）に唯一生息する、専門の技術同人作家。

趣味は 3.5 倍速で聴くオーディオブックの読書と、ジム代わりに 7 年続いているバレエ。

X アカウントは @oukayuka。ブログは <https://klemiuary.com>（くるみ割り書房公式サイト）。

イラスト：黒木めぐみ（くろき・めぐみ）

漫画家、イラストレーター。

「りあクト！」シリーズの表紙イラストを一貫して担当。

りあクト！ TypeScript で始めるつらくない React 開発 第 5 版

【① 言語・環境編】



現場のエンジニアから多大な支持を受ける『りあクト！ TypeScript で始めるつらくない React 開発』の最新 5 版、全 3 部構成の第 1 部「言語・環境編」。

モダンな Web 開発において JavaScript および TypeScript のスキルは大事な基礎体力。その基礎を固めつつ、実際の React 開発において押さえておきたい仕様やさらに進んだ書き方までを学んでいきます。JS や TS が初めての方はもちろん、初～中級者の方にもオススメです。

BOOTH にて絶賛販売中！

(2025 年 5 月 30 日発行／B5 判・214p／電子版 ¥1,200／紙＋電子セット ¥1,550)

【目次】

- 第 1 章 こんにちは React
- 第 2 章 ライトでディープな JavaScript の世界
- 第 3 章 関数型プログラミングでいこう
- 第 4 章 TypeScript で型をご安全に

りあクト！ TypeScript で始めるつらくない React 開発 第 5 版

【② React 基礎編】



現場のエンジニアから多大な支持を受ける『りあクト！ TypeScript で始めるつらくない React 開発』の最新 5 版、全 3 部構成の第 2 部「React 基礎編」。

本質を理解するため歴史を深堀りして、なぜ React が今のようになつたかを解き明かします。そのうえで JSX やコンポーネント、Hooks を学ぶため、理解度が格段にちがってくるはず。初～中級者の方にもオススメです。BOOTH にて絶賛販売中！

(2025 年 5 月 30 日発行／B5 判・216p／電子版 ¥1,200／紙＋電子セット ¥1,550)

【目次】

- 第 5 章 JSX で UI を表現する
- 第 6 章 React をめぐるモダン Web 開発の歴史（前編）
- 第 7 章 ビルドツールでプロジェクトを運用する
- 第 8 章 コンポーネントの基本を学ぶ
- 第 9 章 リンターとフォーマッターでコード美人に
- 第 10 章 React をめぐるモダン Web 開発の歴史（後編）

メールマガジン登録のご案内



くるみ割り書房 BOOTH ページ

「りあクト！」シリーズを刊行している「くるみ割り書房」では、新刊や紙の本の再版予定、その他読者の方へのお知らせなどをメールマガジン形式で不定期に配信しています。

購読をご希望の方は、「くるみ割り書房 BOOTH」で検索して最初に表示された上記画像のページにて、「フォロー」ボタンからサークルのフォロー登録をお願いします。BOOTH の一斉送信メッセージにてメルマガの内容をお届けします。

(※ BOOTH のアカウントが必要になります。BOOTH はピクシブ株式会社が運営するオンラインショップです)

りあクト！ TypeScript で始めるつらくない React 開発 第5版 【③ React 実践編】

2025 年 5 月 30 日	初版第 1 刷発行
2025 年 5 月 30 日	電子版バージョン 1.1.0

著	者	大岡 由佳	
発	行	元	くるみ割り書房
連	絡	先	oukayuka@gmail.com

本書を無断で複写・複製、転載、データ配信、オークションやフリマアプリに出品することを固く禁じます。

© 2025 Yuka O'oka / Klemiuary Books